UNIVERSIDADE DO ESTADO DE SANTA CATARINA – UDESC
CENTRO DE CIÊNCIAS TECNOLÓGICAS – CCT
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO – BCC

KAUAN HENRIQUE WERLICH

PROPOSTA DE UMA ABORDAGEM RAG PARA A RECUPERAÇÃO DE INFORMAÇÕES DE CULTIVOS AGRÍCOLAS DE SC EXTRAÍDAS DO PORTAL EPAGRITEC POR MEIO DE WEB SCRAPING

JOINVILLE
2025

KAUAN HENRIQUE WERLICH

PROPOSTA DE UMA ABORDAGEM RAG PARA A RECUPERAÇÃO DE INFORMAÇÕES DE CULTIVOS AGRÍCOLAS DE SC EXTRAÍDAS DO PORTAL EPAGRITEC POR MEIO DE WEB SCRAPING

Trabalho de conclusão de curso submetido à Universidade do Estado de Santa Catarina como parte dos requisitos para a obtenção do grau de Bacharel em Ciência da Computação.

Orientador: Fabiano Baldo

JOINVILLE
2025

KAUAN HENRIQUE WERLICH

PROPOSTA DE UMA ABORDAGEM RAG PARA A RECUPERAÇÃO DE INFORMAÇÕES DE CULTIVOS AGRÍCOLAS DE SC EXTRAÍDAS DO PORTAL EPAGRITEC POR MEIO DE WEB SCRAPING

Trabalho de conclusão de curso submetido à Universidade do Estado de Santa Catarina como parte dos requisitos para a obtenção do grau de Bacharel em Ciência da Computação.

Orientador: Fabiano Baldo

BANCA EXAMINADORA:

Fabiano Baldo - Doutor
UDESC

Membros:

Rebeca Schroeder Freitas - Doutora
UDESC

Yuri Kaszubowski Lopes - Doutor
UDESC

Joinville, 17 de novembro de 2025

AGRADECIMENTOS

Primeiramente, agradeço aos meus pais e também a minha irmã, que me apoiaram ao longo de toda esta graduação.

Igualmente, agradeço a todo o pessoal do grupo de Banco de Dados e Engenharia de Software (BDES-UDESC), em especial ao professores Fabiano Baldo, Indianara Amâncio e Rodrigo Monzani, que tornaram esse trabalho possível.

Não poderia de deixar de agradecer a toda a equipe do NEXT (Núcleo de Estudos em Xadrez & Tecnologias) pela camaradagem e todas as partidas desses últimos anos.

Agradeço também o apoio financeiro da Fundação de Amparo à Pesquisa e Inovação do Estado de Santa Catarina (FAPESC), Edital 60/2024.

RESUMO

Apesar da abundante quantidade de informações disponibilizadas na Web, é notória a necessidade de uma curadoria e análise criteriosa antes de utilizar qualquer informação retornada em uma busca na Web. A partir do advento dos Modelos de Linguagem em Larga Escala (LLM), foi introduzida uma nova maneira de busca por informações na Web, viabilizada pelas ferramentas de Geração Aumentada de Recuperação (RAG), como, por exemplo, o ChatGPT. Entretanto, estas ferramentas, apesar de possibilitarem uma interação em linguagem natural, o que torna a realização de perguntas muito mais prática e humanizada, ainda sofrem com problemas análogos aos das pesquisas com motores de busca tradicionais, que estão relacionadas à falta de curadoria nas fontes de dados e, adicionalmente, a sua característica de gerar respostas a partir de suas próprias inferências, o que se caracteriza como alucinação. Portanto, para evitar tanto problemas com as fontes de dados quanto a geração de alucinação, uma alternativa adequada é a utilização de fontes de informações curadas contidas em portais ou plataformas de domínio restrito compostas por materiais publicados a partir de revisões por pares. No contexto da agricultora catarinense, o portal Web EpagriTec se apresenta como uma fonte de informações confiável e acurada para consulta, entretanto, ele carece de ferramentas avançadas para a busca por seu conteúdo, nos moldes de uma ferramenta de *Chatbot*. Portanto, este trabalho tem por objetivo desenvolver uma abordagem para geração de respostas concisas e coerentes a perguntas de usuários sobre informações do contexto agrícola contidas no portal EpagriTech, utilizando a técnica de Geração Aumentada por Recuperação (RAG). A avaliação foi conduzida sobre o tópico que trata da cultura do "tomate" dentro do EpariTec, que contém 968 documentos. Sobre esse tópico foram aplicadas 10 perguntas produzidas por um especialista da área. O desempenho da geração da resposta foi comparado entre os LLMs Llama-3-8B e Gemma-7B, analisado estatisticamente por meio do teste de Wilcoxon e medido pela métrica BERTScore. Os resultados demonstram que o *pipeline* de recuperação alcançou 100% de Hit Rate@K geral e 100% de Precisão para 9 das 10 perguntas. O Gemma obteve F1-Score e Precisão estatisticamente superiores, enquanto o Llama foi superior em Recall. Portanto, é possível concluir que a abordagem RAG é viável, e a escolha do LLM implica um *trade-off* entre a precisão do Gemma e a completude (Recall) do Llama.

Palavras-chave: Web Scraping, Modelos de Linguagem em Larga Escala (LLMs), Geração Aumentada por Recuperação (RAG), Pré-processamento de Dados, Vetorização, Armazenamento Vetorial, Agricultura.

ABSTRACT

Despite the abundant amount of information available on the Web, there is a notable need for curation and careful analysis before using any information returned in a Web search. With the advent of Large Language Models (LLMs), a new way of searching for information on the Web was introduced, enabled by Retrieval-Augmented Generation (RAG) tools, such as ChatGPT. However, these tools, despite enabling natural language interaction, which makes asking questions much more practical and humanized, still suffer from problems analogous to those of traditional search engine queries, which are related to the lack of curation in data sources and, additionally, their characteristic of generating responses from their own inferences, which is characterized as hallucination. Therefore, to avoid both problems with data sources and the generation of hallucinations, a suitable alternative is the use of curated information sources contained in portals or restricted-domain platforms composed of materials published through peer review. In the context of Santa Catarina agriculture, the EpagriTec Web portal presents itself as a reliable and accurate source of information for consultation; however, it lacks advanced tools for searching its content in the manner of a Chatbot tool. Therefore, this work aims to develop an approach for generating concise and coherent responses to user questions about information from the agricultural context contained in the EpagriTech portal, using the Retrieval-Augmented Generation (RAG) technique. The evaluation was conducted on the topic dealing with "tomato" cultivation within EpagriTec, which contains 968 documents. Ten questions produced by an expert in the field were applied to this topic. The response generation performance was compared between the Llama-3-8B and Gemma-7B LLMs, statistically analyzed using the Wilcoxon test and measured by the BERTScore metric. The results demonstrate that the retrieval pipeline achieved 100% overall Hit Rate@K and 100% Precision for 9 of the 10 questions. Gemma obtained statistically superior F1-Score and Precision, while Llama was superior in Recall. Therefore, it is possible to conclude that the RAG approach is viable, and the choice of LLM implies a trade-off between Gemma's precision and Llama's completeness (Recall).

Keywords: Web Scraping, Large Language Models (LLMs), Retrieval-Augmented Generation (RAG); Data Preprocessing, Embedding, Vector Storage, Agriculture.

LISTA DE ILUSTRAÇÕES

Figura 1 – Etapas de Web Scraping. 17
Figura 2 – Parte inicial (Fase de Ingestão) do *pipeline* proposto. 35
Figura 3 – Parte final (Fase de Inferência) do *pipeline* proposto. 40

LISTA DE TABELAS

Tabela 1 – Resumo dos Trabalhos Relacionados. 32
Tabela 2 – Conjunto de perguntas (*queries*) utilizadas para a avaliação do pipeline de recuperação. 46
Tabela 3 – Chunks trazidos pelo *pipeline* como contexto logo antes da etapa (ix). . . . 49
Tabela 4 – Resultados Detalhados do BERTScore por Pergunta e Modelo. 50
Tabela 5 – Resultados Detalhados do XLM-RoBERTa por Pergunta e Modelo. 50
Tabela 6 – Comparação Geral: Teste de Wilcoxon (BERTimbau vs. XLM-RoBERTa) . 50
Tabela 7 – Comparação Interna (XLM-RoBERTa): Teste de Wilcoxon (Llama versus Gemma) 51
Tabela 8 – Comparação Interna (BERTimbau): Teste de Wilcoxon (Llama versus Gemma) 51
Tabela 9 – Amostra de resultados do BERTScore do Llama para XLM-RoBERTa e BERTimbau 54
Tabela 10 – Amostra de resultados do BERTScore do Gemma para XLM-RoBERTa e BERTimbau 55

LISTA DE ABREVIATURAS E SIGLAS

BERT	*Bidirectional Encoder Representations from Transformers*
EPAGRI	Empresa de Pesquisa Agropecuária e Extensão Rural de Santa Catarina
FAISS	*Facebook AI Similarity Search*
GPU	Unidade de Processamento Gráfico
LLM	Modelos de Linguagem em Larga Escala
NLP	Processamento de Linguagem Natural
PLM	Modelos de Linguagem Pré-Treinados
RAG	*Retrieval-Augmented Generation*
VRAM	Memória de Acesso Aleatório de Vídeo
WS	*Web Scraping*

SUMÁRIO

1 INTRODUÇÃO . . . 11
1.1 OBJETIVO GERAL . . . 13
1.2 OBJETIVOS ESPECÍFICOS . . . 13
1.3 METODOLOGIA . . . 14
1.4 ESTRUTURA DO TRABALHO . . . 14
2 FUNDAMENTAÇÃO TEÓRICA . . . 15
2.1 MODELOS DE LINGUAGEM EM LARGA ESCALA . . . 15
2.2 EXTRAÇÃO DE DADOS VIA WEB SCRAPING . . . 16
2.3 PRÉ-PROCESSAMENTO DE TEXTO . . . 18
2.4 FRAGMENTAÇÃO DE DOCUMENTOS (*CHUNKING*) . . . 19
2.5 VETORIZAÇÃO E CONSTRUÇÃO DO BANCO VETORIAL . . . 20
2.5.1 Geração de *Embeddings* . . . 20
2.5.2 Armazenamento Vetorial . . . 21
2.6 PROCESSO DE RAG . . . 22
2.6.1 O *Pipeline* do RAG . . . 23
2.6.2 Recuperação em Duas Etapas e *Reranking* . . . 24
2.7 MÉTRICAS DE AVALIAÇÃO . . . 24
2.7.1 Métricas de Recuperação de Informação . . . 24
2.7.2 Métricas de qualidade de Geração de Texto . . . 25
2.8 TRABALHOS RELACIONADOS . . . 27
2.8.1 Considerações sobre os Trabalhos Relacionados . . . 29
2.9 CONSIDERAÇÕES SOBRE O CAPÍTULO . . . 33
3 DESENVOLVIMENTO . . . 34
3.1 PROPOSTA . . . 34
3.2 DESENVOLVIMENTO DO *PIPELINE* – FASE DE INGESTÃO . . . 35
3.2.1 Coleta de Dados por *Web Scraping* . . . 35
3.2.2 Pré-processamento e Armazenamento dos Documentos . . . 36
3.2.3 *Chunking* e Vetorização . . . 37
3.2.4 Indexação Vetorial . . . 38
3.3 DESENVOLVIMENTO DO *PIPELINE* – FASE DE INFERÊNCIA . . . 39
3.3.1 Recuperação Primária e Decisão de Reranqueamento . . . 39
3.3.2 Reranqueamento com Cross-Encoder . . . 40
3.3.3 Filtragem Secundária e Construção do Contexto . . . 41
3.3.4 Aumento e Geração da Resposta . . . 41
3.4 IMPLEMENTAÇÃO . . . 42

4 EXPERIMENTOS E ANÁLISE DOS RESULTADOS 45
4.1 METODOLOGIA DE AVALIAÇÃO **45**
4.2 RESULTADOS **48**
4.2.1 Avaliação da Recuperação de Documentos 48
4.2.2 Análise da Geração de Respostas 49
4.3 CONSIDERAÇÕES SOBRE OS RESULTADOS **56**
5 CONSIDERAÇÕES FINAIS 57
5.1 LIMITAÇÕES **57**
5.2 TRABALHOS FUTUROS **58**
REFERÊNCIAS 59
APÊNDICE A – *SCRIPT* DE INSTALAÇÃO DAS BIBLIOTECAS . . 65

1 INTRODUÇÃO

Segundo o Instituto Brasileiro de Geografia e Estatística (IBGE), o Produto Interno Bruto (PIB) do Brasil alcançou R$10,9 trilhões em 2023, um crescimento recorde em relação aos últimos anos (IBGE, 2024). O setor cujo desempenho apresentou maior destaque foi a agropecuária: com uma participação no PIB de R$677,6 bilhões, o resultado foi 15,1% maior do que no ano anterior. De acordo com o Ministério da Agricultura e Pecuária, o aumento significativo na produção de *commodities* impulsionou esse resultado positivo (BRASIL, 2024).

Uma das razões para esse sucesso é a crescente modernização da cadeia de produção agropecuária. A agropecuária é uma das áreas produtivas cuja eficiência mais aumentou graças à mecanização, inovação e informatização advindas das Revoluções Industriais e Digitais. Já no contexto atual, no qual é amplamente considerado que a Indústria 4.0 está revolucionando diversas áreas (KAGERMANN; WAHLSTER; HELBIG, 2013), os principais avanços ocorrem por meio de abordagens como a Agricultura de Precisão. A Agricultura de Precisão é uma abordagem de gestão agrícola que usa Tecnologia da Informação (TI) para otimizar o uso de recursos e reduzir impactos ambientais (DAS et al., 2018; OLIVEIRA; SILVA, 2023). Outras características importantes são a implementação de técnicas e tecnologias para melhorar a tomada de decisão, a automatização de tarefas e a obtenção de informações relevantes por meio da aquisição de dados. As áreas de pesquisa relacionadas à utilização desses termos estão sendo denominadas atualmente como Agricultura Digital ou Agricultura Inteligente dependendo do contexto (BASSOI et al., 2019).

Para implementar efetivamente essas estratégias, uma grande quantidade de informações é necessária aos profissionais da área, que envolvem especificações técnicas (como concentração de fertilizantes e água, distância entre sementes no plantio, detalhes sobre o solo, quantidade adequada de irrigação, etc.) diferentes para cada cultura a ser manejada. No Brasil, as informações de diversas culturas e regiões específicas já foram pesquisadas e catalogadas em livros e relatórios técnicos por empresas como a Empresa Brasileira de Pesquisa Agropecuária (EMBRAPA) ou a Empresa de Pesquisa Agropecuária e Extensão Rural de Santa Catarina (EPAGRI). Boa parte destas informações estão hoje disponíveis gratuitamente em diversos repositórios na Internet.

Contudo, a forma de busca por essas informações é predominantemente realizada por meio de motores de busca (*search engines*) como Google, DuckDuckGo ou Bing. No entanto, a utilização dessas ferramentas apresenta certas limitações, pois a busca pode retornar documentos extensos que contêm apenas menções às palavras-chave utilizadas, bem como enfrentar dificuldades para mostrar resultados relevantes para itens com interpretações ambíguas.

Paralelamente, nos últimos anos os Modelos de Linguagem em Larga Escala (LLMs - *Large Language Models*), que têm alta capacidade de geração de respostas complexas e coerentes, se tornaram disponíveis e utilizados pela população em geral (como ChatGPT, Gemini, Llama). Modelos de Linguagem (*Language Models*) focam em compreender a distribuição da probabilidade de uma palavra em uma linguagem para que o modelo possa gerar um novo texto

(ANNEPAKA; PAKRAY, 2025). Os LLMs são Modelos de Linguagem que se distinguem pela sua capacidade de realizar várias tarefas de Processamento de Linguagem Natural, incluindo gerar e classificar textos (ANNEPAKA; PAKRAY, 2025).

Portanto, uma das principais inovações introduzidas às *search engines* por meio da incorporação de LLMs foi a possibilidade de utilizar técnicas de Geração Aumentada por Recuperação (RAG) para sumarização de conteúdos, não só em páginas da Web, para a geração de respostas concisas a partir de perguntas descritivas de usuários. Isso permitiu fornecer ao usuário respostas mais concisas do que nas pesquisas tradicionais, e as respostas geradas geralmente apresentam bom nível de acerto para perguntas generalistas. Entretanto, quando perguntas técnicas são feitas, esses LLMs podem responder com base no próprio treinamento ou em fontes externas imprecisas, com conteúdos de propósito geral, pouco técnicos e pouco embasados em referências científicas. Mesmo que existam fontes especializadas públicas na Web, sua relevância pode ser diluída por fatores como: a popularidade da página, foco em respostas amplamente validadas (de consenso geral entre várias páginas), a data de publicação da página e até mesmo a forma como a página está estruturada. Inclusive, algumas páginas utilizam o processo conhecido como Otimização de Conteúdo (incorporando de forma estratégica palavras-chave e HTML Semântico) para torná-la melhor ranqueada. Como consequência, o motor de busca prioriza páginas com essas características, o que torna limitada a sua capacidade de responder correta e satisfatoriamente em contextos específicos, como a agropecuária.

Nesse sentido, empresas públicas de apoio agropecuário brasileiras oferecem uma ampla gama de conteúdos técnicos e com embasamento científico sobre o manejo de culturas, controle de pragas, doenças, plantas daninhas e gestão do solo, disponibilizados em seus portais técnicos e repositórios digitais, tais como, a plataforma EpagriTec. Contudo, apesar de seu valor, o conteúdo dessa plataforma está disposto em páginas Web não otimizadas para motores de busca, o que torna a busca por conteúdos disponibilizados nela de difícil localização. Além disso, seus dados não estão nativamente estruturados para o consumo por um LLM, e suas páginas apresentam problemas de usabilidade até para a navegação humana. Entretanto, apesar destas limitações na estruturação e apresentação do conteúdo, estas páginas se apresentam como uma base de conhecimento relevante para auxiliar usuários na obtenção de respostas a dúvidas relacionadas a temas agropecuários específicos de Santa Catarina. No entanto, para utilizar técnicas mais recentes de busca semântica e geração personalizada de respostas possibilitadas por um LLM, é necessário torná-la apta a ser consumida por esse tipo de solução. Portanto, para que seja possível consultar essa base de dados utilizando a abordagem conhecida como Geração Aumentada de Recuperação (RAG), seus documentos precisam ser primeiro extraídos da Web, limpos, divididos e indexados de forma eficiente. Após isso, é necessário criar um fluxo de busca por documentos, avaliação dos documentos recuperados para identificar se eles são suficientes para a resposta da pergunta realizada e, finalmente, a geração de um *prompt* adequado à geração da resposta esperada.

Portanto, com base no exposto, este trabalho tem a seguinte pergunta de pesquisa: Como

extrair, formatar e estruturar os dados disponíveis no portal EpagriTec de forma a torná-lo apto a responder perguntas realizadas por técnicos em agronomia para auxiliá-los na tomada de decisão em qualquer dúvida sobre o plantio e manejo de culturas? O estudo assume que a extração é possibilitada por meio de técnicas de *Web Scraping* (raspagem de dados); a formatação e armazenamento para busca por similaridade de conteúdo é possibilitada por meio de bancos de dados vetoriais; e a recuperação contextualizada e geração de respostas concisas e coerentes é possibilitada por meio da técnica RAG combinada com LLMs.

O trabalho foi viabilizado pela parceria entre o grupo de Banco de Dados e Engenharia de Software (BDES-UDESC) e a Empresa de Pesquisa Agropecuária e Extensão Rural de Santa Catarina (EPAGRI), na qual a EPAGRI viabilizou a utilização dos dados públicos da plataforma EpagriTec [1], além de prestar apoio na delimitação do escopo e objetivo da pesquisa.

1.1 OBJETIVO GERAL

O objetivo geral deste trabalho é desenvolver uma abordagem para geração de respostas concisas e coerentes a perguntas de usuários sobre informações do contexto agrícola contidas no portal EpagriTec, utilizando a técnica de Geração Aumentada por Recuperação (RAG). Para tal, é necessário arquitetar uma estratégia de automação da extração, formatação, armazenamento e recuperação dos dados da plataforma, em combinação com técnicas para a sumarização e geração da resposta de forma concisa e coerente com a pergunta realizada.

1.2 OBJETIVOS ESPECÍFICOS

Os objetivos específicos são:

- Avaliar a literatura existente sobre técnicas de *Web Scraping* utilizadas na criação de *datasets* (conjuntos de dados); técnicas de pré-processamento e enriquecimento de dados, técnicas de *Data Curation* (Curadoria de Dados) para análise de qualidade dos dados; e arquiteturas de Geração Aumentada por Recuperação (RAG) para a criação de bases de conhecimento contextuais. Com base nisso, decidir quais etapas serão necessárias à criação de um *dataset*.
- Investigar e definir as metodologias de extração (*Web Scraping*) e pré-processamento (limpeza e *chunking* semântico) para a construção de um *dataset* a partir dos dados não estruturados do portal EpagriTec, preservando o significado da hierarquia original do portal.
- Desenvolver um *pipeline* (i.e., um fluxo automatizado de etapas) para a extração, pré-processamento, *chunking*, vetorização e indexação dos dados do portal EpagriTec em um banco vetorial para o consumo por um LLM.

[1] Disponível em: <https://sistemas.epagri.sc.gov.br/sedimob/consulta.action?subFuncao=epagriTec>

- Implementar uma arquitetura RAG, utilizando um *pipeline* de reranqueamento condicional (*conditional reranking*) com *Cross-Encoders*, para melhorar a precisão e a relevância do contexto fornecido ao LLM para a resposta da pergunta realizada.
- Avaliar o desempenho do *pipeline* proposto a partir de um conjunto de consultas-teste no domínio agrícola em duas frentes: (i) utilizando métricas de Recuperação de Informação para mensurar a eficácia da recuperação de documentos pelo Banco de Dados vetorial; (ii) e métricas de similaridade semântica para avaliar a qualidade da geração de respostas pelos LLMs.

1.3 METODOLOGIA

O procedimento metodológico deste trabalho inicia-se com a revisão da literatura científica para determinar o estado atual do conhecimento acerca do assunto. A pesquisa foca nos conceitos relevantes ao tema, incluindo Modelos de Linguagem em Larga Escala (LLMs), a arquitetura de Geração Aumentada por Recuperação (RAG), extração de dados com *Web Scraping* (WS), e os processos de pré-processamento de texto, vetorização e indexação vetorial. Além disso, faz-se uma análise dos trabalhos relacionados que aplicam essas técnicas em domínios de conhecimento específico, como a agricultura.

Na sequência, definem-se: a maneira para formatar e armazenar os dados; o *pipeline* proposto; os conjuntos de dados avaliados; e os parâmetros utilizados no *pipeline*. Em seguida, são avaliadas as técnicas necessárias na automação da extração de dados e em sua integração com um LLM via RAG, considerando fatores como a estruturação e a hierarquia dos dados e das páginas Web do site alvo deste trabalho.

Por último, realiza-se uma série de experimentos para validar a eficácia da abordagem proposta. A metodologia de avaliação baseia-se na definição de um conjunto de perguntas teste de domínio.

1.4 ESTRUTURA DO TRABALHO

Este trabalho divide-se da seguinte forma: No Capítulo 2 revisam-se os principais conceitos necessários à compreensão do trabalho e apresenta os trabalhos relacionados. O Capítulo 3 apresenta a proposta do *pipeline* de ingestão e inferência. Já o Capítulo 4 apresenta os resultados obtidos e a análise deles. Finalmente, o Capítulo 5 apresenta as considerações finais sobre o trabalho.

2 FUNDAMENTAÇÃO TEÓRICA

Neste capítulo descrevem-se os conceitos fundamentais à compreensão deste trabalho. Serão explorados os conceitos relacionados à extração de dados de páginas Web e sua aplicação em LLMs. Além disso, será feita uma análise dos trabalhos relacionados ao tema.

2.1 MODELOS DE LINGUAGEM EM LARGA ESCALA

Modelos de Linguagem em Larga Escala (LLMs - *Large Language Models*) é um termo cunhado para se referir a Modelos de Linguagem Pré-Treinados (PLMs - *Pre-Trained Language Models*) com grande quantidade de parâmetros (i.e., os "pesos" (*weights*) internos que a rede neural ajusta durante o treinamento para aprender os padrões da linguagem), cuja quantidade pode chegar a dezenas ou centenas de bilhões (ZHAO et al., 2025). Recentemente, os LLMs ganharam proeminência e destaque especialmente após o lançamento do GPT-3 (2020) e do ChatGPT (2022) pela empresa OpenAI. O treinamento e a inferência de PLMs seguem o paradigma de pré-treinamento e ajuste fino (*pre-training* e *fine-tuning*), no qual modelos de linguagem com redes neurais recorrentes ou na arquitetura *Transformer* são pré-treinados em grandes corpora (conjuntos) de texto não rotulado da Web, para realizar tarefas como predição de palavras, e então são ajustados com pequenas quantidades de dados rotulados de tarefas específicas (MINAEE; MIKOLOV et al., 2024).

A arquitetura *Transformer*, proposta por Vaswani et al. (2017), é a base da maioria dos LLMs modernos e substituiu redes neurais recorrentes (RNNs) em diversas aplicações de NLP. A principal inovação da arquitetura *Transformer* é o mecanismo de atenção, que permite processar todos os *tokens* de entrada em paralelo e atribuir pesos diferenciados a partes relevantes do texto, melhorando tanto a eficiência quanto a captura de dependências contextuais (VASWANI et al., 2017). Para entender esses modelos, é útil definir suas três propriedades centrais:

- **Parâmetros (*Weights*)**: São os pesos internos que a rede neural ajusta durante o treinamento. Representam o conhecimento aprendido pelo modelo. A quantidade de parâmetros é a métrica mais comum para medir o tamanho de um LLM.

- **Camadas (*Layers*)**: A arquitetura é composta por múltiplas camadas empilhadas. Cada camada processa a informação da camada anterior, permitindo ao modelo aprender representações cada vez mais complexas. O número de camadas define a "profundidade" do modelo.

- **Dimensão Interna (*d_model* ou *hidden size*)**: Define a "largura" do modelo. É o tamanho do vetor que representa cada *token* (palavra ou sub-palavra) enquanto ele é processado internamente pelas camadas.

A combinação da profundidade (número de camadas) e da largura (dimensão interna) é o que resulta na grande quantidade de parâmetros que define um LLM. É importante notar

que a “larga escala” desses modelos resulta em uma grande complexidade computacional: É impraticável executar as operações matemáticas necessárias (multiplicações de matrizes e vetores) para em Unidades de Processamento Central (CPUs) em tempo hábil. Isso ocorre não apenas pela falta de poder de processamento paralelo, mas também pelo gargalo na largura de banda da memória (RAM). Por essa razão, o treinamento e uso de Transformers modernos depende de Unidades de Processamento Gráfico (GPUs) (LI et al., 2025). As GPUs são projetadas para computação paralela e utilizam memória dedicada de alta velocidade (VRAM - Memória de Acesso Aleatório de Vídeo), o que torna viável o processamento de LLMs com essa magnitude de parâmetros.

2.2 EXTRAÇÃO DE DADOS VIA WEB SCRAPING

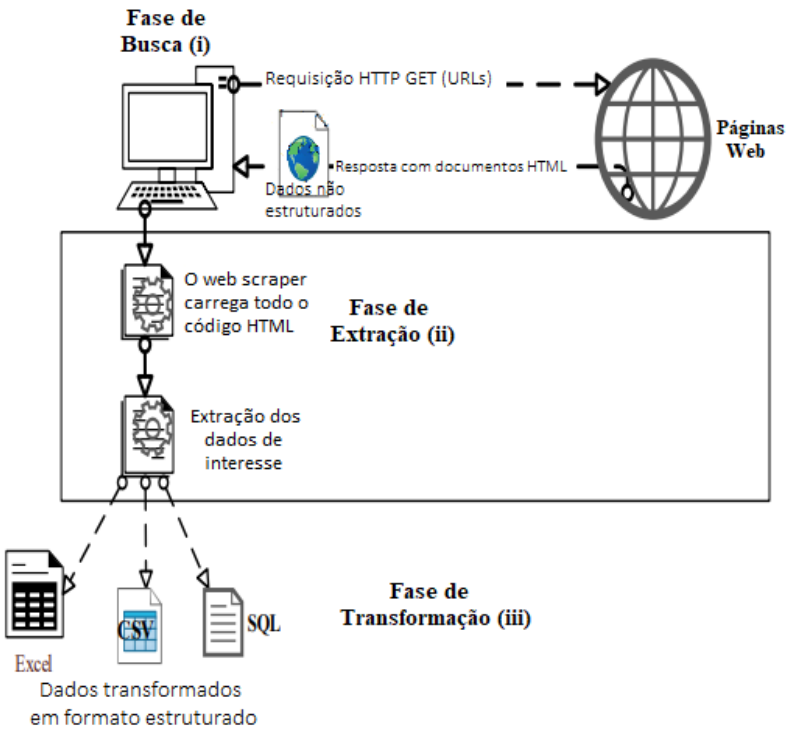
Raspagem da Web (WS - *Web Scraping*) é todo o processo de extrair dados de páginas Web (LAWSON, 2015). O *Web Scraping* automatiza a coleta de informações de sites. Até o surgimento das APIs (*Application Programming Interface*), que são interfaces feitas para tornar a comunicação entre aplicações e servidores mais fácil, WS era a única maneira de obter informações da Internet (KHDER, 2021). Contudo, mesmo as páginas Web que oferecem APIs não são todas gratuitas e também são limitadas em relação a quantos e quais dados elas oferecem. Muitos desenvolvedores priorizam a manutenção do lado do cliente (*front-end*) e não dão a mesma atenção à API. Por isso, não se pode depender de APIs para adquirir dados da Internet, e é preciso aplicar estratégias de WS para consegui-los (MITCHELL, 2018; KHDER, 2021).

De acordo com Persson (2019), as etapas do WS podem ser divididas em 3 fases. A seguir detalham-se essas etapas, ilustradas na Figura 1:

- **Fase de Busca** (etapa (i) da Figura 1): Nesta etapa, o site desejado, que contém as informações ou dados relevantes, é acessado por meio do protocolo HTTP. Um programa de computador (*scraper*), i.e., um software que simula a navegação humana para extrair dados de páginas web, envia requisições GET para o servidor e recebe uma resposta correspondente, que geralmente é um documento HTML. Ferramentas como cURL ou wget podem ser utilizadas para enviar essas requisições HTTP GET ao URL desejado, obtendo o documento HTML de volta na resposta. Isso simula a forma como os navegadores acessam o conteúdo de páginas web. O *scraper* emula um usuário da web, navegando pelos sites.

- **Fase de Extração** (etapa (ii) da Figura 1): Após a obtenção do documento HTML, esta é a fase em que os dados de interesse são extraídos. As tecnologias comumente empregadas nesta fase incluem expressões regulares, bibliotecas de análise HTML (HTML *parsing libraries*) ou consultas XPath (XML Path) para localizar informações dentro do documento.

Figura 1 – Etapas de Web Scraping.



Fonte: Adaptado de: Mustapha et al. (2024, p. 291).

- **Fase de Transformação** (etapa (iii) da Figura 1): Finalmente, os dados de interesse que foram extraídos são transformados em uma versão estruturada. Essa transformação é feita para que os dados possam ser armazenados em formatos como CSV, SQL ou Excel.

Após a extração, também é comum aplicar procedimentos de limpeza e refinamento nos dados brutos, removendo ruído, espaços desnecessários, símbolos ou *tags* residuais. "Sem a limpeza adequada, transformando e estruturando os dados antes da análise, não se pode esperar encontrar padrões significativos" (RAMYA; SHREEDHARA; KAVITHA, 2011). A limpeza e o pré-processamento dos dados extraídos da web são etapas indispensáveis, que exigem a análise manual da estrutura original do conteúdo coletado, a fim de identificar padrões, remover ruídos e garantir a consistência dos dados para análise posterior (TARANNUM, 2019).

Além disso, a estrutura inconsistente é um dos maiores entraves a um processo de WS confiável. Segundo (WEERASINGHE, 2024), os métodos de WS tradicionais enfrentam desafios em lidar com *websites* dinâmicos, em superar medidas *anti-scraping* e em extrair dados estruturados de páginas Web não estruturadas. Nesse contexto, um dos principais desafios teóricos e práticos do *Web Scraping* reside na ética de acesso e na mitigação de sobrecarga de servidores. A literatura define o conceito de "polidez" (*politeness*) como a implementação de restrições voluntárias na taxa de requisições, visando simular o comportamento humano e evitar a Negação de Serviço (DoS) não intencional no servidor-alvo (MITCHELL, 2018). Além das questões éticas, estratégias de limitação de taxa (rate limiting) são respostas técnicas

necessárias para contornar mecanismos de defesa (*anti-scraping*), como bloqueios de IP e *timeouts*, que servidores modernos empregam para identificar e barrar coletas automatizadas agressivas (MITCHELL, 2018).

Em resumo, o WS automatiza a coleta de dados estruturados ou semi-estruturados por meio de *scripts* capazes de acessar páginas Web, interpretar seu HTML e extrair os elementos desejados. Essa técnica se mostra valiosa especialmente em domínios onde APIs não estão disponíveis ou onde as informações estão dispersas (e.g., portais Web ou blogs).

2.3 PRÉ-PROCESSAMENTO DE TEXTO

O processo de pré-processamento de texto envolve a transformação de textos brutos em unidades que podem ser processadas por modelos computacionais, como redes neurais, e é frequentemente o primeiro módulo em um *pipeline* de Processamento de Linguagem Natural (NLP - *Natural Language Processing*) (CAMACHO-COLLADOS; PILEHVAR, 2018).

Segundo Elov, Khamroeva e Khusainova (2023), em um *pipeline* de NLP típico, as etapas iniciais englobam coleta de dados e limpeza do texto. Nesse contexto, o pré-processamento de texto transforma dados brutos em unidades padronizadas para os modelos de linguagem. Upadhye (2020) reforça que identificar e eliminar ruídos (caracteres supérfluos) e padronizar formatos textuais são passos críticos, pois servem de precursor a tarefas como análise de sentimento, classificação de texto e recuperação de informação. No caso de textos extraídos da *Web*, isso inclui remover o chamado *boilerplate* (*scripts*, menus, anúncios etc.), que, segundo Fernández-Pichel et al. (2024), "empobrece o desempenho" de sistemas *downstream* devido à sua natureza repetida. Além disso, corrigem-se questões de codificação (UTF-8), removem-se duplicatas e normalizam-se convenções como pontuação, uso de maiúsculas/minúsculas e acentuação (para garantir consistência) (UPADHYE, 2020).

As etapas abaixo são recorrentes no pré-processamento de textos:

- **Remoção de ruído e sanitização**: eliminar elementos não informativos (ex.: *tags* HTML, *scripts*, anúncios, menus), pontuação irrelevante ou caracteres não-alfanuméricos (FERNÁNDEZ-PICHEL et al., 2024), e corrigir erros de codificação e de digitação. Por exemplo, Upadhye (2020) destaca que erros tipográficos, ortográficos e abreviações não padronizadas degradam a performance de tarefas de NLP, justificando a aplicação de corretores ortográficos e de normalização.

- **Normalização de texto**: padronizar o texto em letras minúsculas (*lowercasing*), remover ou uniformizar acentos e normalizar formatos de data ou outros padrões; também costuma-se eliminar *stopwords* (palavras de baixa carga semântica), o que reduz ruído e a dimensionalidade dos dados (UPADHYE, 2020). Técnicas como lematização também podem ser aplicadas para agrupar formas variantes de palavras comuns.

- **Remoção de duplicatas**: eliminar repetições exatas ou quase-exatas de trechos textuais evita vieses e redundância no corpus (conjunto de documentos) final.

- **Estruturação de registros**: organizar cada segmento de texto limpo como um registro padronizado (e.g., *chunks* ou documentos) acompanhado de metadados (origem, data, tópicos). Murtaza et al. (2025) ilustram essa etapa convertendo dados tabulares em *strings* JSON agrupadas por conceito e dividindo documentos de políticas (como apólices de seguro e políticas para lidar com as informações de clientes) em *chunks* vetorizáveis. Esse processo garante um corpus coerente e indexável para recuperação de informação subsequente.

Seguindo essas etapas, produz-se um corpus confiável e bem estruturado, pronto para indexação e uso em *pipelines* de recuperação desses dados.

2.4 FRAGMENTAÇÃO DE DOCUMENTOS (*CHUNKING*)

Após o pré-processamento e a limpeza textual (Seção 2.3), os documentos raramente são vetorizados por inteiro. A etapa de fragmentação (do inglês, *chunking*) é o processo de dividir grandes documentos de texto em pedaços menores e coesos. Essa etapa é fundamental em *pipelines* RAG por três razões principais:

- **Limites de Contexto:** A maioria dos modelos de *embedding* (técnica detalhada na Seção 2.5) possui um limite máximo de *tokens* de entrada. Por exemplo, muitos modelos da família BERT, que são amplamente utilizados, têm um limite de 512 *tokens* (DEVLIN et al., 2019). Enviar textos maiores resultaria em truncamento e perda de informação.

- **Custo e Latência:** Processar *embeddings* de textos muito longos é computacionalmente mais caro e lento.

- **Relevância na Recuperação:** O objetivo do RAG é encontrar trechos de informação altamente relevantes para a pergunta do usuário. Caso o *chunk* seja muito extenso, a informação específica pode ficar "perdida no meio" (*lost in the middle*), diluída por parágrafos irrelevantes. Isso degrada a qualidade da recuperação (LIU et al., 2023).

Existem diversas estratégias de fragmentação. A mais simples é a fragmentação por tamanho fixo (*fixed-size chunking*), que divide o texto em blocos de *N tokens*, muitas vezes com uma sobreposição (*overlap*) para manter o contexto entre os *chunks*. Embora o tamanho exato seja um hiperparâmetro, é prática comum definir essa sobreposição como um percentual do tamanho total do *chunk*, frequentemente variando entre 10% e 20%, para garantir que o contexto não seja perdido entre os segmentos (Weaviate, 2025). Abordagens mais avançadas incluem a fragmentação recursiva (*recursive chunking*), que tenta dividir o texto com base em separadores

semânticos (como parágrafos e pontuação), ou a fragmentação semântica, que agrupa sentenças com base em sua similaridade de *embedding* (LANGCHAIN, 2024).

Nesse contexto, a escolha de um tamanho de *chunk* de aproximadamente 500 *tokens* é uma prática comum na literatura. Ela equilibra duas necessidades: (i) Compatibilidade, criando uma margem de segurança para modelos clássicos com limite de 512 *tokens*; e (ii) Densidade de Informação, garantindo que o *chunk* recuperado seja conciso e relevante para a consulta do usuário, evitando o problema do "*lost in the middle*" (LIU et al., 2023).

2.5 VETORIZAÇÃO E CONSTRUÇÃO DO BANCO VETORIAL

Esta seção detalha a construção de um Banco de Dados Vetorial. A Subseção 2.5.1 aborda a Geração de *Embeddings*, o processo de conversão de texto em vetores numéricos que capturam o significado semântico. Em seguida, a Subseção 2.5.2 descreve o Armazenamento Vetorial, que foca em como indexar eficientemente esses vetores para permitir buscas rápidas por similaridade.

2.5.1 Geração de *Embeddings*

Na etapa de vetorização, o texto, que é compreensível para humanos, precisa ser traduzido para um formato matemático processável pelo modelo. Cada documento ou fragmento textual é convertido em um vetor numérico de dimensão fixa, conhecido como *embedding*. Esses vetores são um conjunto de coordenadas em um espaço vetorial (e.g., o espaço de 3 dimensões X, Y, Z tem apenas 3 coordenadas) que definem a posição de um objeto no espaço vetorial, e ao capturar aspectos semânticos do texto tornam possível sua comparação matemática com outras representações vetoriais.

Modelos baseados em *transformers*, como o BERT (*Bidirectional Encoder Representations from Transformers*), revolucionaram o campo de NLP ao introduzirem representações contextuais bidirecionais, ou seja, cada palavra em uma sentença é representada levando em conta o contexto à sua esquerda e à sua direita simultaneamente (DEVLIN et al., 2019). Contudo, o BERT original não foi projetado para gerar um vetor fixo para uma sentença completa, o que limita sua aplicação direta em tarefas como busca semântica de documentos. Para comparar sentenças com BERT seria necessário processar cada par de forma independente, o que se torna inviável para grandes coleções.

Para resolver essa limitação, Reimers e Gurevych (2019) propuseram o Sentence-BERT (SBERT), que adapta a arquitetura do BERT para tarefas de similaridade entre sentenças. O SBERT consiste em um modelo siamesa (uma arquitetura do tipo *Bi-Encoder*), no qual duas cópias do BERT compartilham pesos e processam sentenças de forma independente, produzindo *embeddings* vetoriais diretamente comparáveis. Após o processamento, os vetores resultantes podem ser comparados com métricas como a similaridade de cosseno. Esta métrica é usada para medir o ângulo entre dois vetores no espaço de alta dimensão; um ângulo pequeno (cosseno próximo de 1) indica que os vetores apontam para direções semânticas similares, significando

que os textos são relevantes entre si. Isso possibilita operações rápidas de busca por vizinhança em grandes corpora (REIMERS; GUREVYCH, 2019).

O uso do SBERT reduz drasticamente o custo computacional em aplicações como recuperação de documentos. Enquanto o BERT exige recalcular *embeddings* para cada par de sentenças, o SBERT permite calcular os *embedding* uma única vez e reutilizá-los para múltiplas comparações. Por exemplo, a tarefa de encontrar os pares mais semelhantes entre 10 mil sentenças, que levaria aproximadamente 65 horas usando o BERT completo, pode ser realizada em apenas 5 segundos com SBERT, sem perda de precisão nos resultados (REIMERS; GUREVYCH, 2019).

Além do SBERT, outros modelos de geração de embeddings disponíveis pela plataforma *Hugging Face* ou biblioteca *Sentence Transformers* incluem variantes otimizadas para várias línguas, *embeddings* compactos (como `all-MiniLM-L6-v2`) e modelos otimizados para tarefas de recuperação semântica. Todos esses modelos produzem vetores de dimensão fixa, tipicamente 384, 768 ou 1024.

Em suma, esses *embeddings* densos permitem representar sentenças ou documentos por vetores de dimensão fixa que podem ser indexados e posicionados a textos semanticamente semelhantes próximos uns dos outros em um espaço vetorial. Com isso, um trecho de consulta (e.g., pergunta do usuário) pode ser comparado via similaridade de cosseno aos vetores dos documentos, identificando os textos mais relevantes.

2.5.2 Armazenamento Vetorial

Após gerar os *embeddings*, eles são indexados em um banco de dados vetorial, projetado para buscas eficientes por similaridade em espaços de alta dimensão. Bancos de dados vetoriais armazenam grandes coleções de vetores e oferecem busca de vizinhança aproximada (ANN – *Approximate Nearest Neighbor*). Segundo Pan, Wang e Li (2024), as duas estratégias de ANN mais populares divergem na forma como organizam os vetores:

- **Quantização Vetorial**: Esta abordagem, conhecida como *Inverted File Index* (IVF), funciona agrupando os vetores em *clusters* (grupos) baseados em sua similaridade. Primeiro identificam-se os *clusters* mais relevantes para a consulta então compara-se o vetor de busca apenas com os vetores dentro desses grupos, ignorando a vasta maioria do banco de dados.

- **Grafos de Vizinhança (e.g., HNSW - *Hierarchical Navigable Small World*)**: Constrói um grafo onde os vetores (nós) são conectados aos seus vizinhos mais próximos em múltiplas camadas hierárquicas. Uma busca "navega" por esse grafo, começando de um ponto de entrada e "pulando" de nó em nó, aproximando-se logaritmicamente do vizinho mais próximo. Essa estratégia difere de uma mais simples como busca binária pela natureza dos dados, já que não é possível "ordenar" vetores do mesmo jeito que se ordena inteiros.

Outras técnicas comuns de indexação incluem *hashing* e estruturas em árvore (PAN; WANG; LI, 2024). Essas técnicas equilibram desempenho e precisão de busca, permitindo que milhões ou bilhões de vetores sejam consultados com latência reduzida, ao custo de uma pequena perda de precisão.

Além das estratégias de busca, um aspecto fundamental dos bancos vetoriais é a sua capacidade de lidar com a modificação dos dados (inserção, atualização e remoção), conhecida como indexação incremental ou dinâmica. Em um índice puramente estático, qualquer mudança nos dados exige uma reconstrução completa (reindexação) de todo o conjunto de vetores. Já índices dinâmicos são projetados para permitir a adição de novos vetores. Isso é possível porque o *embedder* ranqueia cada *chunk* separadamente, como discutido na Seção 2.5.1. Contudo, a remoção e a atualização são operações mais complexas, especialmente em estruturas de grafos como o HNSW, onde remover um nó pode invalidar a conectividade do grafo. Frequentemente, a remoção é implementada como "*soft delete*": o vetor é marcado como inválido em um filtro separado, mas não é fisicamente removido do índice, o que exige uma filtragem posterior e pode adicionar latência à consulta. O uso contínuo de inserções e remoções incrementais pode levar à degradação do índice (*index degradation*), tornando a busca menos eficiente ou precisa ao longo do tempo. Por isso, muitas estratégias de manutenção adotam uma abordagem híbrida: aceitam atualizações incrementais, mas executam uma reconstrução periódica para otimizar a estrutura do índice.

Diversos *frameworks* implementam essas abordagens, como FAISS e Milvus, além de extensões vetoriais em motores como Lucene ou Elasticsearch. Esses sistemas permitem recuperar, com alta eficiência, os k-vizinhos mais próximos de um vetor de consulta com base em métricas como distância Euclidiana ou produto interno.

Essa infraestrutura é essencial na recuperação de contexto: quando o LLM recebe uma pergunta, ela é convertida em vetor e comparada aos vetores armazenados no banco vetorial. As passagens mais semelhantes são então recuperadas e incorporadas ao contexto da resposta, permitindo que o LLM produza respostas fundamentadas em informações relevantes e atualizadas do domínio. Esse processo caracteriza o RAG, fundamentado na próxima seção.

2.6 PROCESSO DE RAG

A técnica de Geração Aumentada de Recuperação (RAG – do inglês *Retrieval-Augmented Generation*) consiste em recuperar pedaços (*chunks*) de documentos relevantes de uma base de conhecimento externa, por meio de cálculo semântico de similaridade (GAO et al., 2024). Em RAG, o LLM é complementado por um índice vetorial denso de documentos externos, em que cada vetor representa um trecho de um documento. Esse arranjo permite que informações atualizadas de fontes externas sejam acessadas no momento da geração do texto, aumentando a precisão e a credibilidade das respostas em tarefas intensivas em conhecimento. Gao et al. (2024) ressaltam que o RAG incorpora dados de bases de conhecimento especializadas,

elevando a exatidão das respostas e permitindo atualizações contínuas de domínio específico. Em outras palavras, o RAG mitiga limitações dos modelos puramente paramétricos (controlados por parâmetros) como as "alucinações" ao adicionar uma camada de busca documental, tornando o modelo adaptável a novos conhecimentos.

Além disso, embora dados estruturados possam fornecer respostas muito precisas (devido à clareza de seus metadados), sua integração em RAG envolve *trade-offs* entre custo de ingestão e ganho de desempenho em produção (MURTAZA et al., 2025). Dados não estruturados podem ser indexados diretamente em um banco vetorial, e basta buscar os trechos relevantes e fornecê-los ao LLM, o que reduz o número de chamadas ao modelo e mantém a latência baixa. Já dados estruturados não se encaixam tão bem nesse fluxo. Ou gera-se dinamicamente uma consulta, o que incorre em aumento de chamadas ao LLM, de custo e de latência; ou converte-se o próprio registro em texto para vetorização antes da inferência, o que eleva o trabalho inicial de pré-processamento. Murtaza et al. (2025) realiza a conversão de dados estruturados em arquivos JSON e posteriormente os vetoriza como texto não estruturado, e apesar de conseguirem bons resultados, reconhecem que isso aumenta o tempo de pré-processamento.

2.6.1 O *Pipeline* do RAG

A arquitetura de RAG envolve vários componentes integrados, e o processo é tipicamente dividido em duas fases principais: a Fase de Ingestão (ou Indexação), um processo offline (i.e., realizado no *background* da aplicação) de preparação dos dados, geralmente processando-os em grandes volumes; e a Fase de Inferência, o processo online (i.e., respostas imediatas de acordo com a necessidade) que gera a resposta ao usuário.

Além da obtenção de dados, a Fase de Ingestão envolve técnicas previamente discutidas: todas as etapas do pré-processamento (Seção 2.3, a fragmentação (Seção 2.4), a vetorização (Subseção 2.5.1) e o armazenamento em um banco de dados vetorial (Subseção 2.5.2).

Já a Fase de Inferência envolve:

1. **(Pré-etapa) Vetorização da Consulta:** A pergunta do usuário é convertida em um vetor usando o mesmo *Bi-Encoder* da Fase de Ingestão.

2. **R – Recuperação (*Retrieval*):** Usa-se o vetor da consulta para buscar no banco vetorial, recuperando os *k chunks* de documentos mais semanticamente similares à pergunta.

3. **A – Aumento (*Augmentation*):** Os textos originais dos *chunks* recuperados (o "contexto") são reunidos e combinados com a pergunta original do usuário, formando um *prompt* detalhado.

4. **G – Geração (*Generation*):** O *prompt* aumentado (pergunta + contexto) é enviado ao LLM, que gera a resposta final fundamentada no contexto fornecido.

Essa separação entre Ingestão e Inferência é um dos principais benefícios do RAG, e permite que a base de conhecimento seja atualizada, expandida ou re-indexada (e.g, com novos

modelos de *embedding* ou com mais dados) de forma assíncrona, sem interromper o fluxo de inferência em tempo real, garantindo a manutenibilidade e a escalabilidade do sistema.

2.6.2 Recuperação em Duas Etapas e *Reranking*

A etapa de Recuperação depende de como o sistema compara a pergunta do usuário com os documentos. Para isso, existem duas arquiteturas principais de *encoders* baseados em Transformers, que precisam ser diferenciadas:

1. **Bi-Encoder (Codificador Duplo):** Gera *embeddings separados* para a pergunta e para o documento. O SBERT (discutido na Subseção 2.5.1) é um exemplo de Bi-Encoder. A comparação (feita por similaridade, e.g., distância de cosseno ou distância euclidiana) é feita depois que os vetores estão prontos. É extremamente rápido e escalável, ideal para a busca inicial em grandes conjuntos de documentos.

2. **Cross-Encoder (Codificador Cruzado):** Não gera vetores separados. Ela recebe a pergunta e o documento *juntos* como uma entrada única e processa ambos ao mesmo tempo. Isso permite que o mecanismo de atenção do *Transformer* avalie a interação direta entre os *tokens* da pergunta e do documento, resultando em uma pontuação de relevância muito mais precisa, mas é computacionalmente muito mais lento, pois precisa reprocessar cada par (pergunta-documento).

Embora a busca vetorial com *Bi-Encoders* seja extremamente rápida e escalável, ela realiza uma recuperação de "grão grosseiro" (i.e. *coarse-grained*), organizando as informações em categorias gerais (HUMEAU et al., 2020). Na prática, o *Bi-Encoder* é excelente para encontrar um conjunto de k documentos provavelmente relevantes, mas não é otimizada para definir qual deles é o mais relevante.

Para solucionar essa limitação, arquiteturas RAG de alta precisão empregam um *pipeline* de recuperação em duas etapas: primeiro usa um Recuperador (*Retriever*) rápido e depois usa um Reranqueador (*Reranker*) preciso (THAKUR et al., 2021).

2.7 MÉTRICAS DE AVALIAÇÃO

A avaliação de um *pipeline* de Geração Aumentada por Recuperação (RAG) é intrinsecamente complexa, pois envolve duas partes distintas que precisam ser mensuradas: a eficácia da Recuperação de Informação (*Retrieval*), discutida na Subseção 2.7.1; e a qualidade da Geração de Texto (*Generation*), discutida na Subseção 2.7.2.

2.7.1 Métricas de Recuperação de Informação

Para a Recuperação de Informação, as métricas clássicas são:

- **Precisão (*Precision*)**: Mede a proporção de *chunks* recuperados que são de fato relevantes. É a métrica de quão correta é a lista de resultados. É definida pela contagem de Verdadeiros Positivos (TP - documentos relevantes recuperados) e Falsos Positivos (FP - documentos irrelevantes recuperados). Conforme definida em Manning, Raghavan e Schütze (2009), a precisão é dada por:

$$\text{Precisão} = \frac{TP}{TP+FP}$$

- **Cobertura (*Recall*):** Mede a proporção de todos os documentos relevantes existentes no corpus que foram efetivamente recuperados pelo sistema pela contagem dos Verdadeiros Positivos ((TP) e dos Falsos Negativos (FN - documentos relevantes não recuperados). Conforme Manning, Raghavan e Schütze (2009), a cobertura é dada por:

$$\text{Cobertura} = \frac{TP}{TP+FN}$$

- **Taxa de Acerto (Hit Rate@K)**: A métrica Hit Rate@K é uma abordagem pragmática, que mede se pelo menos um item relevante aparece dentro das k primeiras posições recuperadas. Como definida por Li et al. (2020), se um ou mais itens relevantes estão na lista dos top k, é considerado um acerto (*Hit*), com valor igual a 1. Se nenhum item relevante estiver na lista dos top k, é considerado uma falha, com valor igual a 0. Ao fim mede-se a porcentagem de acertos em relação ao total de perguntas feitas:

$$\text{Hit Rate@}K = \frac{\text{Número de Perguntas com Acerto}}{\text{Número Total de Perguntas}}$$

Herlocker et al. (2004) afirma que a métrica de *recall* pura é academicamente inviável porque, para calcular Falsos Negativos (FNs), o avaliador precisaria ter conhecimento "onisciente" de todo o corpus para identificar todos os Falsos Negativos que o sistema deixou de trazer. Isso é relevante, pois LLMs são capazes de usar partes incompletas da informação ou com conteúdo similar para montar suas respostas, e sem verificar o corpus completo não é possível calcular a cobertura. Quando se usa estratégia de *chunking*, é preciso levar em conta não apenas os documentos, mas também os *chunks* relevantes em cada documento (partes de um mesmo documento podem ser de temas diferentes).

Por causa disso a métrica de Hit Rate@K comumente substitui a cobertura em sistemas de recuperação. Outra motivo para isso é que se existirem dois ou mais *chunks* com informações similares, e o LLM obtiver apenas um deles corretamente, ele ainda assim conseguirá responder corretamente, mas a sua pontuação de cobertura ficará prejudicada. O uso de Hit Rate@K ameniza esse problema.

2.7.2 Métricas de qualidade de Geração de Texto

Já para avaliar a Geração de Texto, é preciso mensurar a qualidade da resposta final produzida pelo LLM. Diferentemente de tarefas com respostas binárias, a geração de linguagem

natural possui uma intrínseca subjetividade: uma resposta pode estar correta factualmente, mas diferir lexicamente (uso de palavras diferentes) da resposta de referência (*ground truth*). As métricas de geração podem ser categorizadas em três abordagens principais: baseadas em n-gramas, baseadas em *embeddings* e baseadas em modelos treinados (*Model-based*).

As métricas baseadas em n-gramas avaliam a sobreposição lexical entre a resposta gerada e a referência humana. As principais são:

- **BLEU (*Bilingual Evaluation Understudy*)**: Calcula a precisão da contagem de n-gramas (sequências de *n* palavras) que aparecem tanto na resposta gerada quanto na referência. O BLEU aplica uma penalidade de brevidade para evitar que frases muito curtas tenham notas artificialmente altas (PAPINENI et al., 2002).

- **ROUGE (*Recall-Oriented Understudy for Gisting Evaluation*):** Foca na cobertura (*recall*) de n-gramas, sendo amplamente usada em sumarização (LIN, 2004). Variantes comuns incluem o ROUGE-N (n-gramas fixos) e o ROUGE-L, que considera a Mais Longa Subsequência Comum (*Longest Common Subsequence*) entre os textos.

Porém, ambas as métricas falham em capturar o significado semântico, pois dependem da correspondência exata de palavras (*exact matching*) e não reconhecem sinônimos ou paráfrases. Assim, uma resposta correta escrita com vocabulário diferente do gabarito pode receber pontuações baixas tanto em BLEU quanto em ROUGE.

Para superar a dependência da correspondência exata de palavras, métricas semânticas utilizam vetores densos para comparar o significado dos textos. A principal é o BERTScore, métrica utiliza um PLM para gerar embeddings para os tokens da resposta gerada (candidata) e da resposta de referência (*ground truth*). Em seguida, calcula a similaridade de cosseno par-a-par entre os *tokens* para encontrar a melhor correspondência semântica (ZHANG et al., 2020). Isso permite que ela reconheça respostas como semanticamente corretas, mesmo que elas não usem exatamente as mesmas palavras que a referência.

Assim, o BERTScore avalia a similaridade de significado em vez de apenas a forma exata, sendo mais robusto para avaliar paráfrases geradas por LLMs. Contudo, o BERTScore exige a existência de uma resposta de referência. Nos casos em que as respostas de referência não estão disponíveis, a literatura recente propõe o uso de métricas baseadas em modelos treinados ou o paradigma *LLM-as-a-Judge*:

- **BLEURT:** O BLEURT (*Bilingual Evaluation Understudy with Representations from Transformers*) é um modelo de regressão baseado em BERT, treinado em grandes bases de dados de avaliações humanas. Ao contrário das métricas estáticas, o BLEURT aprende a predizer como um humano avaliaria a frase, capturando nuances de fluência e coerência (SELLAM; DAS; PARIKH, 2020).

- **Métricas Específicas de RAG (RAGAS):** Em sistemas RAG, muitas vezes não existe uma "resposta de referência"perfeita. Nesses casos, utiliza-se o paradigma *LLM-as-a-*

Judge (LLM como juiz), onde um modelo avançado (como GPT-4) avalia a resposta com base em critérios definidos (ES et al., 2024).

Em suma, a revisão das métricas de geração evidencia que nenhuma abordagem isolada é suficiente para capturar a complexidade da linguagem natural em sistemas RAG. Enquanto métricas de n-gramas (BLEU, ROUGE) oferecem baixo custo computacional e reprodutibilidade, elas falham na avaliação semântica. Por outro lado, métricas baseadas em *embeddings* (BERTScore) e particularmente modelos avaliadores (RAGAS, BLEURT) apresentam maior correlação com o julgamento humano, mas exigem maior poder computacional. O uso do paradigma *LLM-as-a-Judge* também impõe desafios significativos de reprodutibilidade e custo computacional, visto que depende de chamadas a modelos proprietários. Portanto, esses fatores devem ser considerados na escolha da estratégia de avaliação a ser utilizada.

2.8 TRABALHOS RELACIONADOS

Os trabalhos relacionados foram selecionados com foco em aplicações que envolvem técnicas de Web Scraping, vetorização de documentos e uso de arquiteturas RAG e *fine-tuning* de LLMs (isoladamente ou em conjunto) para recuperação e adaptação inteligente de informações. Esta seção apresenta um resumo da revisão da literatura realizada.

Rezayi et al. (2022) desenvolveram o modelo de linguagem AgriBERT, pré-treinado em uma base de dados textuais de alimentos e agricultura, e o afinam para "*matching*" semântico entre descrições de alimentos e dados nutricionais. O *fine-tuning* desse modelo incluiu o uso de conhecimento externo (a ontologia FoodOn), o que resultou em desempenho superior na tarefa de correspondência alimentar-nutrição.

Em paralelo, Jiang et al. (2025) propuseram o KALLM, um LLM agrícola orientado por conhecimento, onde prepararam um extenso *dataset* (aproximadamente 220 mil pares de perguntas e respostas) a partir de documentos agrícolas. Eles aplicaram um *fine-tuning* coordenado que amplifica *tokens* específicos do domínio agrícola e introduziram um mecanismo RAG auto reflexivo baseado em correspondência de tópicos para aprimorar a recuperação de evidências. Esse *pipeline* de *Fine-tuning* Supervisionado (SFT - *Supervised Fine-tuning*) coordenado com o uso de RAG superou significativamente LLMs de domínio geral em fluência e precisão no contexto agroindustrial.

Panoutsopoulos et al. (2024) também investigaram o *fine-tuning* no domínio agrícola, comparando BERTs gerais e específicos (Agriculture-BERT, SciBERT etc.) para extração de termos técnicos. Eles mostraram que o Agriculture-BERT geralmente supera os demais e que a estratégia de congelar ou atualizar camadas específicas durante o treino influencia fortemente a capacidade de identificar termos já vistos, sinônimos ou termos novos.

Para sistemas de pergunta-resposta (QA) e *chatbots* agrícolas, técnicas de Recuperação-Aumentada por Geração (RAG) têm sido adotadas com destaque. Akbar (2025) avalia empiricamente o efeito de estratégias de reranqueamento e computação em memória em modelos

RAG para QA agrícola de pequena escala. Seu *pipeline* usa Sentence Transformer para *embeddings*, FAISS para busca vetorial e um LLM pré-treinado para gerar respostas. Observou-se que o reranqueamento baseado em classificação *zero-shot* trouxe ganhos marginais, enquanto o uso de FAISS como *embedder* reduziu drasticamente a latência de recuperação, melhorando a adequação do sistema a cenários em tempo real.

Zhang et al. (2025) propuseram o BeefBot, um *chatbot* para pecuaristas que compara diretamente duas estratégias de preparação de um LLM *open-source*: (i) *fine-tuning* de um modelo base (LLaMA 2 13B) com dados agropecuários, e (ii) um *pipeline* RAG. Utilizando um corpus pré-indexado de artigos e relatórios sobre pecuária, aplicaram pré-processamento (remoção de *boilerplate*, tokenização e *chunking* de aproximadamente 500 *tokens*), geraram *embeddings* com Sentence-Transformer (all-MiniLM) e indexaram-nos em FAISS. Na abordagem RAG, consultam o índice para recuperar trechos relevantes que são então fornecidos ao LLaMA2 aplicando a técnica LoRA (*Low-Rank Adaptation*) para geração. Na abordagem de *fine-tuning*, ajustam o LLaMA2 diretamente no corpus. Os testes mostraram que o *pipeline* RAG alcançou 85% de precisão em Q&A, contra 78% do *fine-tuning* puro, além de reduzir em 30% a latência de resposta. Esses resultados indicam que RAG pode superar *fine-tuning* direto em cenários de perguntas agrícolas, oferecendo alta precisão sem depender de grandes volumes de dados anotados.

Glukhikh, Chernysheva e Shentsov (2024) apresentam um sistema de suporte à decisão para estufas inteligentes baseado em LLM com Recuperação Aumentada por Geração (RAG). A arquitetura proposta integra três bases de conhecimento: observações de sensores (um banco estruturado com leituras de temperatura, umidade e detecção de doenças); uma base de doenças e medicamentos e uma base de referência geral. Na sequência, elas são pré-processadas, divididas em *chunks* e vetorizadas por um modelo *Transformer*. Para a recuperação, empregam um esquema híbrido de busca que combina similaridade vetorial e BM25, permitindo selecionar os trechos mais relevantes conforme a consulta do usuário. O módulo RAG injeta esses fragmentos no *prompt* de um agente controlador (implementado sobre Llama2 7B LoRA), que orquestra as ferramentas de busca e gera respostas fundamentadas. Nos experimentos com dados reais de estufa, o sistema demonstrou ser capaz de diagnosticar doenças de plantas e sugerir tratamentos de forma confiável, reduzindo o risco de "alucinações" do modelo ao confirmar quando não dispõem de informação suficiente no contexto.

Ainda no domínio da agricultura digital, Xiong et al. (2025) criaram uma base de conhecimento local em proteção vegetal: vetorizam 7000 artigos científicos e selecionam 568 para gerar dados Q&A (perguntas e respostas) via LLM, resultando em um conjunto de aproximadamente 9000 pares de perguntas e respostas limpos. Em seguida aplicam LoRA para ajustar um modelo InterLM-20B, gerando o InterLM-20B-LoRA integrado ao LangChain e à base vetorial. Crucialmente, incorporam RAG para permitir que o modelo recupere conhecimento específico de campo antes de responder, mitigando alucinações. O sistema resultante atingiu 97% de exatidão nas respostas, evidenciando que a combinação de *fine-tuning* eficiente (LoRA) com

pipeline de recuperação melhora significativamente a precisão em QA agrícola especializado. Em outras palavras, *pipelines* que incluem *embeddings* de documentos de domínio, mecanismos de recuperação e integração com LLMs conseguem respostas mais fidedignas.

Outros trabalhos reforçam a relevância de arquiteturas RAG em diferentes domínios. Por exemplo, Vaghefi et al. (2023) apresentam o ChatClimate, um protótipo de AI conversacional para questões climáticas. Eles conectam o GPT-4 ao relatório AR6 do IPCC (fonte autoritativa de clima) por meio de um *pipeline* de LangChain: o texto foi dividido em trechos, embutido via `text-embedding-ada-002` e indexado para recuperação. Comparando três configurações (GPT-4 puro, GPT-4 consultando somente AR6, e uma versão híbrida GPT-4 + AR6), verificaram que o modelo híbrido gerou respostas com mais exatidão, demonstrando que ancorar o LLM em literatura científica relevante via RAG aumenta a confiabilidade do sistema.

Sohn et al. (2025) introduziram o *pipeline RAG*[2] (Rationale-Guided RAG) para Q&A médico. Eles combinam três inovações: filtragem de trechos informativos com base em "perplexidade de razão", geração de raciocínios pelo LLM para guiar as consultas e recuperação balanceada de quatro corpora biomédicos para mitigar vieses. Esse pipeline estruturado de RAG superou o estado da arte em *benchmarks* médicos, alcançando ganhos de até 6,1% em exatidão.

Já no campo jurídico, Ma et al. (2025) criam o modelo jurídico de linguagem SDD-LawLLM: geram sinteticamente dados estilo "*Chain-of-Thought*" a partir de normas via GPT-4, depois realizam *fine-tuning* em um LLM jurídico e empregam RAG para incorporar informação legal nos contextos. O resultado foi um modelo que supera LLMs genéricos em tarefas do *benchmark* LawBench, atribuindo parte dos avanços ao uso de RAG para contextualizar respostas jurídicas.

Em síntese, os trabalhos revisados que utilizaram RAG enfatizam um fluxo comum: (i) coleta ou uso de dados especializados (literatura, relatórios ou documentos do domínio); (ii) pré-processamento desses textos; (iii) vetorização (embeddings); (iv) indexação (por exemplo, FAISS ou outros bancos vetoriais) para recuperação eficiente; (v) mecanismos de recuperação (ex: similaridade vetorial, reranqueamento) que selecionam trechos relevantes; e (vi) integração com o LLM para geração da resposta. Poucos detalham o uso de WS e observa-se que muitos pressupõe o acesso a bases de dados especializadas. Yang et al. (2024) e Samuel et al. (2025) são trabalhos ainda não publicados (*preprints*) que também fazem uso de um RAG para complementar as respostas de um LLM, mas também não usam WS para obter seus dados. O uso de WS automatizado seria natural para compilar conteúdos atualizados (como notícias agrícolas, boletins técnicos e publicações oficiais) que alimentam essas bases. Em particular, a incorporação de *pipelines* de RAG tem se mostrado eficaz para conter alucinações do modelo e garantir precisão nas respostas, seja no contexto agrícola ou em outras áreas de aplicação.

2.8.1 Considerações sobre os Trabalhos Relacionados

Com base nos trabalhos levantados na Seção 2.8, a Tabela 1 compara os trabalhos com base nos seguintes critérios de análise:

- **Técnica Principal**: verifica como cada estudo aborda a integração entre recuperação de documentos e geração de texto, permitindo identificar quais abordagens priorizam atualização dinâmica versus especialização por *fine-tuning*.
- **Fonte Principal de Dados**: evidencia a natureza e a heterogeneidade do corpus utilizado, como textos agrícolas, relatórios técnicos e mídias multimodais; apontando o grau de independência em relação a bases estáticas ou a coleta automatizada.
- **Pré-Processamento**: expõe as práticas de limpeza e *chunking* aplicadas antes da vetorização, fator determinante para a qualidade dos *embeddings* e a eficiência da busca semântica.
- **Ferramenta de *Embedding***: reflete o método de conversão de texto para vetor, apontando *trade-offs* entre modelo leve versus representação de domínio mais precisa (modelos com mais parâmetros são mais precisos, mas isso eleva o custo computacional).

A análise dos resultados contidos na Tabela 1 revela alguns padrões importantes. Primeiro, percebe-se que na maioria dos trabalhos com RAG utiliza-se *embeddings* gerados por ferramentas comerciais como o ChatGPT e APIs da OpenAI, que, embora pagas (i.e., são ferramentas de propriedade intelectual de alguma empresa, não disponíveis ao público de forma completamente gratuita), oferecem atualmente as representações semânticas mais precisas e avançadas do mercado. Mas isso compromete a reprodutibilidade, a acessibilidade, a transparência e os custos de produção ao adotá-las como base de seus estudos. Dentre as ferramentas listadas nesses trabalhos, as principais alternativas *open-source* e/ou gratuitas foram o Sentence-Transformers (*embedder*), o FAISS (buscas em RAG) e o LLaMA (LLM). Além disso, observa-se variedade de *embedders*, e ausência das etapas de WS.

Quanto ao Pré-Processamento, a maioria dos autores enfatiza a remoção de partes repetidas e o *chunking* em fragmentos de tamanho controlado, prática que se mostra essencial para otimizar tanto a indexação quanto a coerência do contexto recuperado. Por fim, a Fonte de Dados distingue-se pela crescente adoção de *pipelines* que incorporam conteúdo multimodal ou bases especializadas (em domínios específicos, como medicina, agricultura ou direito), enquanto projetos mais antigos ou genéricos podem apoiar-se em corpora estáticos ou APIs de terceiros.

Também percebe-se que o RAG oferece excelentes resultados para a recuperação e geração de informações relevantes. Usar dados provenientes de WS também oferece potencial se comparado ao uso de corpus fixos: Ao extrair conteúdo diretamente de sites e portais agrícolas, o sistema mantém a base de conhecimento sempre atualizada (novas técnicas de cultivo, alertas fitossanitários, boletins climáticos), o que reduz significativamente o risco de respostas desatualizadas e melhora a relevância das recomendações. O módulo de WS também pode ser adaptado para incluir novas fontes, sem alterar o restante do *pipeline* de indexação e geração.

De forma complementar, observa-se na Tabela 1 que, os 10 trabalhos revisados empregam diferentes estratégias de recuperação de informações:

- 2 empregam apenas *fine-tuning* de LLMs (Rezayi et al. (2022) e Panoutsopoulos et al. (2024)).
- 3 combinam *fine-tuning* + RAG (Jiang et al. (2025), Xiong et al. (2025) e (MA et al., 2025)).
- 5 adotam exclusivamente RAG ou variantes (Zhang et al. (2025), Akbar (2025), Glukhikh, Chernysheva e Shentsov (2024), Vaghefi et al. (2023) e Sohn et al. (2025)).

Esse predomínio de RAG em relação ao *fine-tuning* reflete a capacidade do RAG de permitir incorporar informações recentes sem retreinar completamente o modelo, bastando atualizar a base vetorial. Já o *fine-tuning* exige conjuntos de dados grandes e estáticos, além de custosas etapas de re-treinamento para cada novo domínio ou atualização de conteúdo. *Fine-tuning* também requer dados anotados em larga escala (Q&A, relatórios), o que nem sempre está disponível. RAG prescinde de anotações, trabalhando com texto bruto indexado, reduzindo o custo de criação de corpora específicos.

Por esses motivos, os trabalhos recentes privilegiam RAG, ou o combinam com *fine-tuning* seletivo, buscando um equilíbrio entre especialização do modelo e agilidade na incorporação de novos conhecimentos.

Portanto, tendo em vista os pontos levantados, as principais técnicas utilizadas ao longo desse trabalho serão RAG e WS. Por mais que outros trabalhos (notavelmente Zhang et al. (2025)) tenham um escopo parecido ao que será proposto, nenhum deles apresenta WS como etapa de obtenção de dados. Além disso, nenhum deles é um trabalho nacional, e as soluções adotadas não refletem a realidade dos dados de treinamento disponíveis na EPAGRI, o que reforça a necessidade de uma investigação na integração entre WS e RAG.

Tabela 1 – Resumo dos Trabalhos Relacionados.

Trabalho	**Técnica Principal**	**Fonte Principal de Dados**	**Pré-Processamento**	**Ferramenta de Embedding**
Rezayi et al. (2022)	Pré-treino + Fine-tuning	Corpora de texto alimentares; ontologia FoodOn	Limpeza de HTML, remoção de scripts, tokenização	BERT-based embeddings
Vaghefi et al. (2023)	RAG (GPT-4 + AR6 IPCC)	Relatórios IPCC AR6	Chunking em trechos de 1000 tokens, remoção de boilerplate	text-embedding-ada-002
Glukhikh, Chernysheva e Shentsov (2024)	RAG	Leituras de sensores; base de doenças; base referência	Limpeza de dados heterogêneos, chunking	Transformer-based embeddings
Panoutsopoulos et al. (2024)	Fine-tuning de BERT	Corpus anotado de textos agrícolas	Tokenização, lematização, remoção de stopwords	BERT embeddings
Zhang et al. (2025)	RAG / Fine-tuning	Corpus pré-indexado (papers, relatórios)	Limpeza de boilerplate, chunking	SentenceTransformer (all-MiniLM)
Jiang et al. (2025)	Fine-tuning coordenado + RAG	220000 pares Q&A de documentos agrícolas	Filtragem temática, limpeza de ruído	text-embedding-ada-002
Akbar (2025)	RAG + Re-ranking	Conjunto QA agrícola anotado	Normalização, stemming, remoção de duplicatas	SentenceTransformer (all-MiniLM)
Xiong et al. (2025)	LoRA + RAG (QA)	Base vetorizada de artigos de proteção vegetal	Extração de seções, limpeza, chunking	OpenAI/Vicuna embeddings
Sohn et al. (2025)	*RAG*^{2} (com raciocínio)	4 corpora biomédicos	Filtragem por perplexidade, remoção de ruído	BERT-based embeddings
Ma et al. (2025)	Fine-tuning sintético + RAG	Dados legais sintéticos (CoT)	Geração de Chain-of-Thought, chunking	text-embedding-ada-002
Método Proposto	RAG + Web Scraping	Portal agrícola (EpagriTec)	Parsing HTML, remoção de padrões, chunking	BAAI/bge-m3

Fonte: O autor.

2.9 CONSIDERAÇÕES SOBRE O CAPÍTULO

Este capítulo estabeleceu os fundamentos técnicos do *pipeline* RAG, desde a extração de dados via WS até a geração de respostas fundamentadas por LLMs. As seções de pré-processamento, vetorização e processo de RAG forneceram o arcabouço conceitual necessário para compreender cada etapa da proposta. Na revisão das soluções correlatas, identifica-se que, embora haja maturidade em técnicas de RAG e *fine-tuning*, ainda existe espaço para aprofundar o uso de WS como fonte primária, especialmente em contextos regionais como Santa Catarina. Essas observações guiarão a definição do *pipeline* deste trabalho.

3 DESENVOLVIMENTO

Este capítulo apresenta a proposta de um *pipeline* de coleta de dados, pré-processamento, vetorização, armazenamento vetorial e geração de resposta a perguntas de usuários sobre informações contidas na plataforma EpagriTec. A Seção 3.1 detalha a solução proposta. A Seção 3.2 apresenta o fluxo da parte de Ingestão do *pipeline* (incluindo extração, pré-processamento, *chunking*, vetorização e indexação dos dados em um banco vetorial). A seção 3.3 apresenta a parte de Inferência do *pipeline* (arquitetura RAG com reranqueamento condicional) e a Seção 3.4 apresenta a implementação do protótipo de validação do trabalho.

3.1 PROPOSTA

Na medida em que a necessidade de recuperação rápida e eficiente de informações em vários setores tem impulsionado o desenvolvimento de LLMs e *chatbots* especializados em diversos domínios, essa demanda também se faz presente no setor agropecuário.

O EpagriTec é um acervo de informações técnicas sobre cadeias produtivas do estado, tendo como objetivo facilitar aos técnicos de campo e agricultores o acesso, a difusão e a busca do conhecimento agropecuário atual (MORALES; HICKEL, 2023). Ela organiza o conhecimento de cadeias produtivas importantes (como tomate, cebola, arroz irrigado), oferecendo para cada cultura tópicos técnicos como manejo, nutrição, fitossanidade, entre outros. Cada cadeia é subdividida em temas específicos (e.g., adubação, cultivares, controle de pragas), permitindo que o usuário navegue de acordo com sua necessidade. Os conteúdos estão disponibilizados em página Web. Em suma, o portal apresenta informações agrícolas organizadas por cultura e manejo.

Informações como as disponíveis no portal EpagriTec são de grande valor para os profissionais da área, mas as páginas do portal são antigas e apresentam baixa usabilidade, se comparadas ao padrão atual de Desenvolvimento Web.

Primeiramente, o site faz uso de tabelas para dispor listas de navegação (em vez de listas `<ul>` ou `<ol>`), e o conteúdo de texto controlado por *scripts* é exemplo de HTML não-semântico. A estética e padrões de interação são característicos de aplicações Web do início dos anos 2000, e o conteúdo não é apresentado de forma direta: O usuário é forçado a realizar ações repetidamente para conseguir ler um único artigo. Isso quebra o fluxo de leitura e dificulta a leitura rápida da página, principalmente em usuários acostumados ao padrão moderno que esperam que o conteúdo apareça à medida que se rola para baixo (*scroll down*).

Além dessas características, a navegação no portal não é baseada em *links* HTML simples (`<a>`), mas em *JavaScript*. Isso prejudica especialmente a capacidade de motores de busca em usar "rastreadores" (*crawlers*) para navegar no site seguindo *links* para descobrir novas páginas. Logo, mesmo *chatbots* disponíveis em navegadores teriam dificuldade em usar as informações do site.

Somando-se a esses fatores o fato de que o portal não possui uma API pública, e que

metadados cruciais (como as legendas de imagens) estão ocultos em galerias separadas, percebe-se que formatar as informações do EpagriTec para consumo por um LLM (de forma que ele consiga usá-las para responder às perguntas de um usuário) tem grande potencial de utilização.

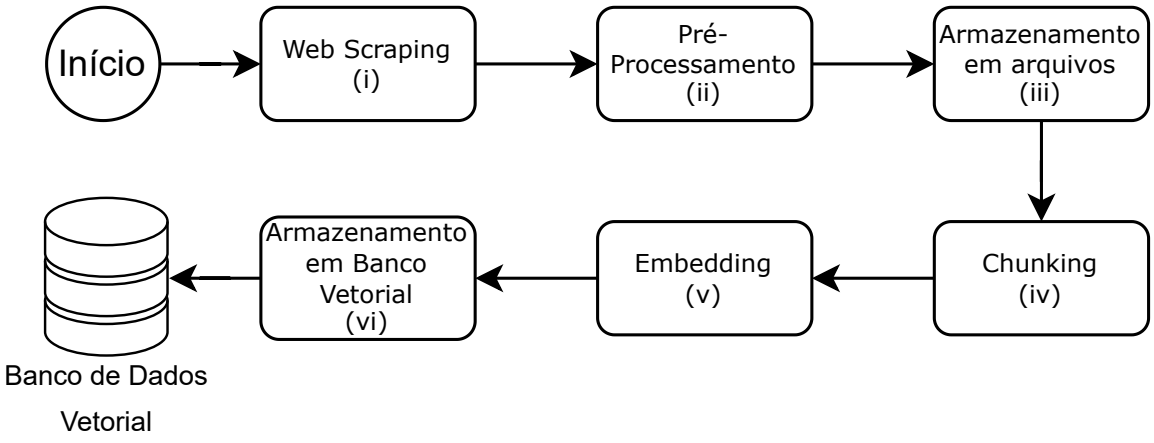
Portanto, com base na revisão dos trabalhos relacionados apresentada na Seção 2.8, entende-se que a proposta do uso de técnicas de *Web Scraping* e *Embedding* para vetorizar e indexar as informações contidas nas páginas no EpagriTec como documentos em um banco de dados vetorial, juntamente com a técnica de RAG para buscar os documentos mais similares e relevantes a pergunta de pesquisa e posterior compilação de uma resposta sumarizada, são suficientes para resolver a demanda de busca por informações agropecuárias contidas no EpagriTec.

As próximas subseções descrevem os fluxos do *pipeline* de ingestão (preparação) e inferência (geração da resposta), nas Seções 3.2 e 3.3, respectivamente.

3.2 DESENVOLVIMENTO DO *PIPELINE* – FASE DE INGESTÃO

A parte inicial do *pipeline* a ser desenvolvido é a Fase de Ingestão, o processo de construir e preparar a base de conhecimento. Ele não depende da interação do usuário e pode ser executada em lote. Essa parte é detalhada na Figura 2, e tem seis etapas principais: (i) coleta de dados especializados (por meio de técnicas de WS); (ii) pré-processamento dos documentos (remoção de padrões, conteúdo irrelevante e limpeza do texto); (iii) armazenamento em arquivos; (iv) *chunking*; (v) *embedding* (vetorização) e (vi) armazenamento no Banco de Dados Vetorial (indexação). Cada uma dessas etapas é detalhada nas subseções subsequentes.

Figura 2 – Parte inicial (Fase de Ingestão) do *pipeline* proposto.



Fonte: O autor.

3.2.1 Coleta de Dados por *Web Scraping*

A etapa inicial, ilustrada pela etapa (i) na Figura 2, se caracteriza pela obtenção dos dados brutos da plataforma EpagriTec. Os dados são extraídos por meio de técnicas de WS. A coleta

de dados foi projetada como um *crawler* recursivo, dividido em quatro fases lógicas:

1. **Inicialização e Otimização**: O processo simula a navegação humana dentro do portal, e instancia o navegador em modo *headless* (sem interface gráfica). Para acelerar a coleta e focar apenas no conteúdo textual, o navegador é configurado para bloquear o *download* de recursos desnecessários, como imagens, fontes e arquivos CSS.

2. **Descoberta de Pontos de Entrada**: O *crawler* não parte de uma página de cultura, mas sim do índice principal do EpagriTec. Ele analisa esta página inicial para extrair a lista de todos os IDs de categorias-raiz (i.e. "Tomate" e "Banana") a partir de chamadas *JavaScript*. Estes IDs formam a "fila" inicial de trabalho. Um conjunto para controlar os IDs que já foram processados também é inicializado para evitar processamento duplicado.

3. **Extração de Conteúdo Dinâmico**: Para cada ID na fila, o *crawler* acessa a URL da página correspondente ao ID. Como o conteúdo do portal é dinâmico, o *script* primeiro simula cliques de usuário em todos os *links* "...mais" para forçar a expansão e carregamento de todo o texto oculto. Somente após a página estar totalmente expandida o HTML completo é salvo localmente (no formato ID.html, i.e. 024.html).

4. **Descoberta de Sub-Páginas (Recursão)** Como a navegação do *site* não é feita por *links* HTML (<a>), o *crawler* inspeciona a página para encontrar a variável *JavaScript* interna `dadosTabela`. Esta variável atua como um "mapa" que lista todas as subseções (filhas) da página atual. O *script* extrai os IDs dessas filhas e chama recursivamente a Fase 3 para cada novo ID encontrado, "descendo" pela hierarquia do *site* até que não haja mais sub-páginas para descobrir.

Esse processo garante a coleta de todas as páginas vinculadas hierarquicamente a partir do código (URL) inicial, e pode ser adaptado para começar a partir de outros pontos da hierarquia.

Além disso, como já foi mencionado, as imagens disponíveis no corpo das páginas do EpagriTec são apenas miniaturas que levam à imagem principal apenas quando se clica nelas, para poupar recursos. Positivamente, essas imagens expandidas costumam ter uma legenda que descreve as imagens, algo muito útil como contexto. Para não perder essas informações, utiliza-se um fluxo paralelo de *scraping*, muito semelhante ao anterior, mas com o objetivo de baixar apenas os HTMLs com as imagens expandidas. Esses HTMLs são salvos em local diferente dos HTMLs das páginas de texto, e são úteis na próxima etapa.

3.2.2 Pré-processamento e Armazenamento dos Documentos

Para cada documento HTML resultante da coleta de dados, inicia-se o pré-processamento dos documentos (etapa (ii) da Figura 2), de acordo com algumas das técnicas investigadas na literatura e detalhadas na seção 2.3:

- Implementa-se uma heurística para identificar e remover os menus do portal, que na ocasião foram identificados como informação supérflua, pois sua tarefa é listar os *links* dos subtópicos de um tema.
- Aplica-se a sanitização de caracteres: Além da remoção padrão de *tags* HTML e *scripts*, remove-se caracteres não UTF-8 e lixo visual típico do portal EpagriTec (e.g., a presença do link "fechar /" em todos os textos expandidos previamente).
- Reconstrói-se a hierarquia semântica do documento: *Tags* de cabeçalho (h1, h2) são convertidas em títulos formatados, e *tags* de negrito (<b>) que funcionam como subtítulos são identificadas e formatadas, preservando a estrutura lógica do texto original.
- Além disso, extrai-se um título hierárquico (conhecido como *breadcrumb*) presente no cabeçalho de cada página (e.g., "Tomate - Pragas - Mosca-branca"). Esse título é concatenado em uma única *string* e inserido no topo do arquivo extraído, junto com a URL de origem.
- As URLs e links de imagens são salvos juntos ao texto, na ordem em que aparecem dentro do texto. Se a imagem for uma das que está salva como um HTML separado (e portanto, contém uma legenda na forma de texto logo acima da imagem), essa legenda é inserida logo abaixo do link da imagem.

O resultado são arquivos limpos, apenas com as informações relevantes. A seguir, poderia-se perfeitamente prosseguir o pré-processamento para armazenar todos os documentos em formato JSON ou *markdown* (.md), mas como já se mencionou na Seção 2.6, a solução mais rápida e escalável é manter os dados de forma não estruturada. Por isso, o armazenamento em arquivos (etapa (iii) da Figura 2) é feito em formato `.txt` no estilo de um arquivo texto enriquecido: o contexto fica salvo junto com os metadados, lembrando a representação visual e sequencial que um usuário teria ao lê-lo na própria página. Os arquivos texto são então salvos em uma pasta, prontos para a vetorização.

3.2.3 *Chunking* e Vetorização

Pelas razões ressaltadas na Seção 2.4, na sequência aplica-se *chunking* aos documentos (etapa (iv) da Figura 2). Cada documento é convertido em um *chunk* com conteúdo em formato texto e com metadados. Os metadados são:

- **Título**: O título hierárquico completo extraído do cabeçalho do arquivo.
- **URL**: A URL de origem da página de conteúdo.
- **cdEstrutura**: O código de identificação da página, extraído do nome do arquivo.
- **Arquivo de origem**: O nome do arquivo `.txt` de onde o *chunk* se originou.

- **Parte**: O número sequencial do *chunk* dentro daquele documento.
- **Total de partes**: O número total de *chunks* gerados para aquele documento.
- **Id**: Um identificador único e rastreável para o *chunk* específico, combinando a origem e o número da parte.

Porém, uma limitação comum da busca vetorial é que esses metadados não são levados em conta pelo *embedder*, e só voltam a aparecer quando um *chunk* é recuperado e consultado. Eles podem ser recuperados posteriormente e inseridos na resposta, mas o *embedder* não os leva em conta na criação do *embedding*. Para corrigir isso, neste trabalho opta-se por inserir o metadado mais importante, o título, no topo de cada *chunk*.

Imediatamente após a criação dos *chunks* é possível seguir à vetorização (etapa (v) da Figura 2). A seguir, extrai-se um *embedding* (representação numérica) de cada item por meio de um *embedder*. Durante a conversão, cada vetor numérico gerado é também matematicamente padronizado (processo conhecido como normalização). Isso garante que todos os vetores tenham a mesma magnitude (comprimento Euclidiano), tornando os cálculos de similaridade na etapa de indexação mais rápidos e consistentes.

3.2.4 Indexação Vetorial

Após a vetorização dos documentos, os vetores gerados são armazenados em uma estrutura indexada otimizada para buscas por similaridade semântica (etapa (vi) da Figura 2), de modo a permitir identificar rapidamente documentos próximos no espaço vetorial. O objetivo principal dessa etapa é permitir que, a partir de uma consulta em linguagem natural, o sistema possa recuperar rapidamente os documentos mais semanticamente relacionados, baseando-se na proximidade dos vetores no espaço latente.

Cada vetor é inserido no índice por meio de lotes (itens), mantendo a consistência entre os identificadores dos documentos e os vetores armazenados. A indexação também preserva metadados relevantes, como o nome do arquivo original e a posição no corpus, permitindo a rastreabilidade dos trechos recuperados. O método de busca adotado é baseado em distância vetorial, retornando os k vetores mais próximos de uma consulta embutida.

O índice vetorial resultante pode ser salvo em disco e reutilizado em execuções futuras. Como o modelo de *embedding* processa cada *chunk* de forma totalmente isolada, ao adicionar um novo documento só é necessário processar os *chunks* desse novo arquivo, gerar os seus vetores e adicioná-los ao índice vetorial. Os vetores antigos permanecem válidos.

O mesmo vale ao modificar ou remover arquivos. Basta remover os vetores antigos, processar os novos e adicioná-los ao índice. No caso da remoção, também remove-se os identificadores daquele arquivo do índice.

Vale a ressalva de que a literatura menciona problemas ao utilizar essas funções de acrescentar e remover documentos várias vezes consecutivas, e o banco de dados vetorial pode

perder a consistência. Por isso, costuma ser padrão refazer o *embedding* de todos os documentos após grandes mudanças. Isso não foi necessário nesse trabalho, já que a base de dados foi a mesma do início ao fim, e portanto ela não sofreu alterações.

3.3 DESENVOLVIMENTO DO *PIPELINE* – FASE DE INFERÊNCIA

A partir do Banco de Dados vetorial feito na fase anterior, é possível prosseguir à Fase de Inferência, processo que começa no momento em que o usuário faz uma pergunta e espera uma resposta. Essa parte é detalhada na Figura 3, e tem dez etapas principais: (i) recuperação de documentos primária; (ii) filtro de qualidade primário; (iii) etapa de decisão para evitar reranqueamento se já existirem documentos dentro do *threshold*; (iv) recuperação de documentos secundária; (v) reranqueamento; (vi) filtro de qualidade secundário; (vii) verificação se existem documentos dentro do *threshold*; (viii) filtragem para enviar apenas um número máximo de *chunks* como contexto; (ix) construção do *prompt* para o LLM; e (x) geração da resposta pelo LLM. Cada uma dessas etapas é explicada e detalhada nas Subseções subsequentes.

3.3.1 Recuperação Primária e Decisão de Reranqueamento

O fluxo de inferência é iniciado quando a consulta do usuário (em linguagem natural) é recebida. Essa consulta é então vetorizada utilizando o mesmo modelo *embedder* empregado durante a Fase de Ingestão, garantindo que a pergunta e os documentos existam no mesmo espaço vetorial.

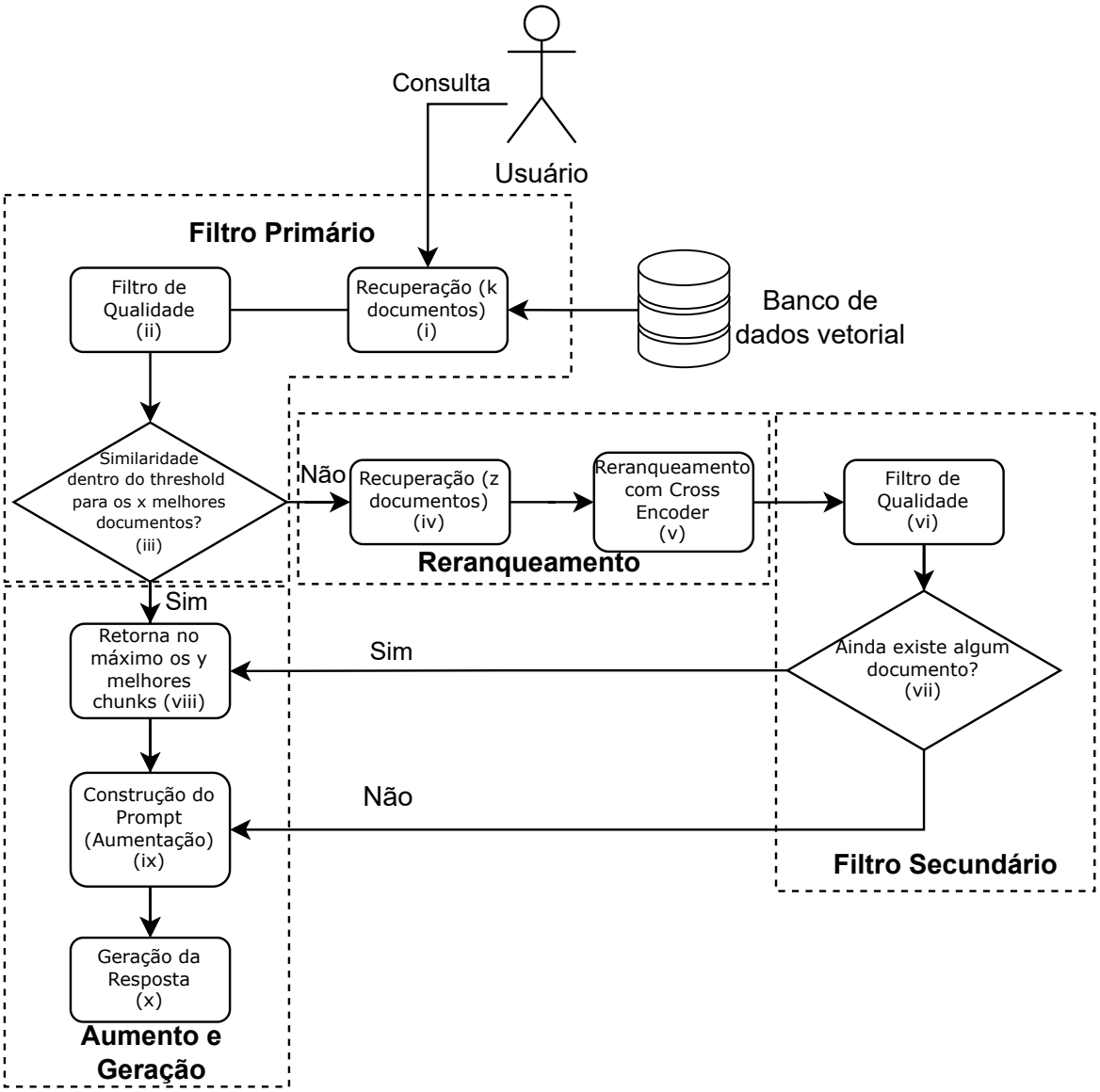
Na sequência, listam-se as próximas etapas de acordo com o *pipeline* da Figura 3:

- **(Etapa i) - Recuperação (k documentos)**: O vetor da consulta é usado para realizar uma busca por similaridade semântica (como a similaridade de cosseno) no banco de dados vetorial. Esta é a recuperação rápida, que retorna rapidamente os k documentos candidatos mais próximos à consulta.

- **(Etapa ii) - Filtro de Qualidade**: Os k documentos recuperados, juntamente com suas pontuações de similaridade, passam por um filtro de qualidade primário, e recebem pontuação de acordo com o critério de similaridade usado.

- **(Etapa iii) - Decisão de Limiar (*Threshold*)**: Avaliam-se as pontuações dos x melhores documentos recuperados (onde $x \leq k$). Se as pontuações dos x melhores documentos forem excepcionalmente altas, ultrapassando um limiar (*threshold*) pré-definido, o sistema considera a recuperação primária como "confiável". Nesse caso (caminho "Sim"), o *pipeline* avança diretamente para a etapa (viii), pulando o reranqueamento. Esta otimização reduz a latência e o custo computacional em consultas fáceis, cuja resposta já é evidente na primeira busca. Caso os x melhores documentos não atinjam o *threshold*, segue-se à etapa iv (caminho "Não").

Figura 3 – Parte final (Fase de Inferência) do *pipeline* proposto.



Fonte: O autor.

Em resumo: A Etapa (i) é a recuperação inicial, a etapa (ii) gera as pontuações e a Etapa (iii) age com base nessas pontuações.

3.3.2 Reranqueamento com Cross-Encoder

Se as pontuações da recuperação primária não atingirem o *thrsehold* (caminho "Não" na etapa iii), o sistema usa reranqueamento para refinar os resultados. A seguir, listam-se as etapas do reranqueamento de acordo com o *pipeline* da Figura 3:

- **(Etapa iv) - Recuperação (z documentos)**: O sistema pode opcionalmente realizar uma segunda recuperação, buscando um conjunto maior de z documentos (onde $z > k$). O

objetivo é aumentar a cobertura, garantindo que documentos relevantes, mas que não ficaram no "*top-k*", sejam considerados pelo reranqueador.

- **(Etapa v) - Reranqueamento com Cross-Encoder**: Os z documentos candidatos são processados por um modelo Cross-Encoder, processando a consulta original e o texto de cada documento candidato simultaneamente. Isso permite uma análise mais precisa e contextualizada, reordenando os documentos com base em uma pontuação mais robusta.

3.3.3 Filtragem Secundária e Construção do Contexto

Na sequência, listam-se as etapas posteriores ao reranqueamento, nas quais os resultados passam por uma seleção final antes de serem enviados ao LLM, conforme as etapas da Figura 3:

- **(Etapa vi) - Filtro de Qualidade**: Os documentos reranqueados passam por um novo filtro de qualidade, que avalia as novas pontuações atribuídas pelo *Cross-Encoder*, de acordo com o critério de similaridade usado.
- **(Etapa vii) - Verificação de Documentos**: O sistema verifica quantos documentos passaram pelo filtro de qualidade. Se nenhum documento for considerado relevante (caminho "Não"), nenhum documento é retornado, e o *pipeline* segue à etapa ix. Se ainda existir algum documento (caminho "Sim"), prossegue-se à etapa viii.
- **(Etapa viii) - Seleção de Contexto (*y* *chunks*)**: Se documentos relevantes forem encontrados, selecionam-se os y melhores documentos. Este passo serve para garantir que a quantidade de texto enviada ao LLM esteja dentro de sua janela de contexto máxima.

3.3.4 Aumento e Geração da Resposta

Estas são as etapas finais, conforme as etapas da Figura 3, que efetivamente correspondem ao "AG" (*Augmented Generation*) da sigla RAG:

- **(Etapa ix) - Construção do *Prompt* (Aumentação)**: Constrói-se o *prompt* que será enviado ao LLM. Este *prompt* é "aumentado", pois combina a pergunta original do usuário com os textos literais dos y *chunks* de documentos relevantes recuperados na etapa anterior. O *prompt* também contém instruções que orientam o LLM a basear sua resposta exclusivamente nas informações fornecidas no contexto e a responder de forma coesa e sucinta. Caso não exista nenhum documento relevante, o LLM é orientado a simplesmente responder "Nenhuma informação relevante encontrada nos contextos fornecidos" (o que seria possível a partir do caminho "Não" da etapa vii).
- **(Etapa x) - Geração da Resposta**: O *prompt* aumentado é enviado ao LLM, que gera a resposta final em linguagem natural para o usuário. Configura-se o LLM para não usar ou usar muito pouco o próprio conhecimento (obtido a partir de seu treinamento), e sim se basear no contexto fornecido.

O fluxo detalhado visa equilibrar a latência com a precisão da resposta, garantindo que o LLM receba o contexto mais relevante possível da base de dados da EpagriTec.

3.4 IMPLEMENTAÇÃO

As etapas do protótipo do *pipeline* proposto foram implementadas em ambientes distintos, dependendo da necessidade ou não de GPU. Como mencionado na Seção 2.1, as tarefas que envolvem o uso de LLMs podem ser aceleradas com o uso de GPU. Como os modelos utilizados na geração das respostas precisam de grandes quantidades de VRAM providas pela GPU, optou-se por utilizar o ambiente do Google Colab para todas as etapas que envolvem LLMs. Utilizou-se o plano gratuito, que oferece acesso a um ambiente com Python versão 3.12 e uma GPU T4, com 15 GB de VRAM útil (de 16 GB totais), aproximadamente 11.5 GB de RAM útil (de 12.7 GB totais) e aproximadamente 74 GB de armazenamento em disco útil (de 112.6 GB totais). As bibliotecas e versões utilizadas no ambiente do Colab se encontram no Apêndice A.

As etapas que não envolvem GPU, especialmente as 3 primeiras (WS, pré-processamento e armazenamento vetorial dos arquivos), foram executadas em estações de trabalho locais. Por serem tarefas que exigem conexão à rede e realizam leitura e escrita dos arquivos, utilizou-se um ambiente com 20 GB de RAM, armazenamento SSD e Python versão 3.10, onde o acesso direto ao sistema de arquivos e à rede é mais estável que no ambiente de nuvem.

A obtenção de dados por meio de *Web Scraping* foi implementada por meio de um *script* Python versão 3.10.4. Essa linguagem é amplamente utilizada no campo de análise e obtenção de dados, e possui amplo suporte a bibliotecas de WS como Selenium. O uso do Selenium permite a automação de diversas tarefas por meio da execução de *JavaScript* diretamente na página Web, e possibilita o uso de duas características chave: o Bloqueio de Recursos (e.g., Imagens e CSS) e o Modo Headless (que evita a renderização da GUI do navegador). Dessa forma, otimiza-se o WS no computador que realiza a consulta ao servidor para obter apenas o texto do site. Além disso, essa biblioteca fornece recursos para expandir seções ocultas de sites e esperar o carregamento de todos os recursos da página antes de iniciar o download do HTML.

São usadas diversas estratégias de espera para evitar erros no processo de *scraping*:

- Aguarda-se por até 10 segundos o carregamento inicial do corpo da página (*body*) e dos elementos de navegação iniciais.
- Usa-se 0,1 segundos de intervalo entre cada expansão (clique) nos botões "...mais", para aguardar um tempo mínimo para o navegador processar o evento de clique sem travar.
- Após terminar de expandir todos os blocos da página, espera-se 1 segundo antes de capturar o HTML e prosseguir à próxima requisição. Isso garante que todo o conteúdo dinâmico seja renderizado visualmente antes de salvar o arquivo.

Após a etapa de pré-processamento, a decisão de manter os arquivos em formato não estruturado (texto bruto no formato `.txt`) também fundamenta-se no fato de que a organização

das informações nas diferentes páginas do portal não seguia um único padrão (e.g., o padrão de escrita e o nível de detalhes varia entre seções de diferentes cultivares como banana, tomate e cebola). Em experimentos realizados na etapa inicial do trabalho, essas informações trouxeram pouco valor e aumentaram o ruído já mencionado na literatura ao usar informações estruturadas em um processo de RAG (Seção 2.6).

Para cada documento de texto, divide-se o texto em *chunks* de 500 *tokens*, com uma janela de sobreposição de 80 *tokens*, valores esses que seguem o padrão da literatura, conforme a Seção 2.4.

Para converter cada item em um *embedding*, utilizou-se o modelo `BAAI/bge-m3` da biblioteca Sentence-Transformers, acessado por meio da interface HuggingFaceEmbeddings no *framework* LangChain. A escolha foi motivada por ele ser um modelo *open source* de alto desempenho, classificado entre os melhores em *benchmarks* de recuperação de informação multilíngue (como MTEB (*Massive Text Embedding Benchmark*), MLDR (*Multilingual Document Retrieval*) e MKQA (*Multilingual Question Answering*)). O bge-m3 possui aproximadamente 568 milhões de parâmetros, dimensão interna e de saída de tamanho 1024, 24 camadas e tamanho máximo de entrada de até 8192 *tokens*. Além disso, o bge-m3 é construído a partir do XLM-Roberta, um modelo de linguagem pré-treinado pela empresa Meta em texto de mais de 100 idiomas, o que o torna multilíngue (CHEN et al., 2024). Adicionalmente, o *embedder* é configurado para que todos os vetores gerados sejam normalizados. Esta normalização padroniza a magnitude dos vetores, tornando os cálculos de similaridade na etapa de indexação mais eficientes.

A indexação foi realizada com a biblioteca FAISS (*Facebook AI Similarity Search*), amplamente utilizada na literatura (como mencionado na Seção 2.8). A biblioteca FAISS é desenvolvida pela empresa Meta AI (antigo Facebook AI Research), e é uma ferramenta *open source* com desempenho otimizado para GPU e com busca eficiente de baixa latência que realiza as principais operações de um banco de dados vetorial (JOHNSON; DOUZE; JÉGOU, 2019). Além disso, como com as estratégias utilizadas o número total de *chunks* está apenas na casa dos milhares, é utilizada a estratégia de indexação IndexFlatL2 (busca por força bruta, padrão do FAISS), que traz os *chunks* mais próximos de forma 100% precisa. Existem outras estratégias como IndexIVF (que usa *clusters*) e IndexHNSW (que usa grafos), que têm mais escalabilidade para *chunks* na casa dos milhões, mas ao custo de uma perda de precisão, como discute-se na Seção 2.5.2. O índice também foi configurado para funcionar com persistência em disco, o que permite que seja recarregado em execuções futuras, garantindo reutilização e escalabilidade da arquitetura.

Para a implementação da etapa de reranqueamento, o modelo selecionado foi o BAAI/bge-reranker-large. A escolha deste modelo é estratégica, pois ele é um Cross-Encoder da mesma família BAAI do modelo Bi-Encoder (o bge-m3) utilizado para a geração dos embeddings, o que garante coerência no uso de ferramentas desenvolvidas para serem complementares. A sua capacidade multilíngue também garante um desempenho robusto para o *corpus* em português.

O *pipeline* implementado utiliza a classe HuggingFaceEmbeddings da biblioteca *lang-*

chain_huggingface como interface e o *framework* LangChain para integrar os componentes do *pipeline*, como o carregamento de documentos e a interface com modelos de *embedding*.

Os LLMs utilizados foram quantizados em 4 bits, e configurados com o padrão de criatividade igual a 0,25 (o que deixa o modelo mais fidedigno aos textos recuperados, sem perder a capacidade de gerar uma resposta coerente).

Além disso, como hiperparâmetros do *pipeline* de Inferência foram configurados com os seguintes valores:

- $k = 5$ (etapa (i));
- *threshold* de distância euclidiana menor ou igual a 0,45 (etapa (ii)), o que é equivalente a similaridade de cosseno aproximadamente maior ou igual a 0,90.
- $z = 20$ (etapa (iv));
- *threshold* de distância euclidiana menor ou igual a 0,7 (etapa (vi)), o que é equivalente a similaridade de cosseno aproximadamente maior ou igual a 0,75.
- $y = 5$ (etapa (viii)).

Nesse caso, a equivalência de similaridade de cosseno e distância de cosseno pode ser afirmada, pois foi realizada a normalização dos vetores.

Os valores de k e de y são iguais a 5 para manter consistência com práticas da literatura e da indústria. O valor de z é puramente empírico, enquanto que os *thresholds* foram estabelecidos nesses patamares para garantir como retorno apenas *chunks* muito similares (e.g., a indústria avalia valores menores que 0,7 como alta similaridade). Logo, esses hiperparâmetros foram definidos de maneira empírica, e trouxeram os melhores resultados para essa quantidade de documentos dentre todos os experimentos realizados.

4 EXPERIMENTOS E ANÁLISE DOS RESULTADOS

Este capítulo apresenta os testes realizados e os resultados obtidos com o *pipeline*. Os experimentos focaram em mensurar a eficácia da recuperação de documentos e em avaliar a qualidade das respostas obtidas. A metodologia empregada para a avaliação é descrita na Seção 4.1, enquanto a Seção 4.2 apresenta e analisa os resultados obtidos. Por último, a Seção 4.3 apresenta as considerações sobre os resultados.

4.1 METODOLOGIA DE AVALIAÇÃO

Como a qualidade das respostas geradas pelo *pipeline* depende diretamente da relevância dos documentos recuperados do banco vetorial, a primeira parte das avaliações se concentrou em medir a eficácia na recuperação dos arquivos. Portanto, os experimentos simulam as perguntas de um usuário da EpagriTec. Para isso, contou-se com a ajuda de um especialista da área, o Doutor em agronomia Rodrigo Martins Monzani (professor do Instituto Federal Catarinense (IFC)).

A ajuda do especialista possibilitou a criação de dez perguntas pertinentes aos dados contidos no portal EpagriTec, que tiveram seus documentos relevantes manualmente identificados e rotulados, o que garantiu a realização dos experimentos. Para este trabalho, foi escolhido o tópico que trata da cultura do "tomate" no portal EpagriTec, que sozinho soma um total de 968 documentos.

Para avaliar a recuperação de documentos, optou-se por utilizar as métricas de precisão e Hit Rate@K, devido à dificuldade em utilizar cobertura (de acordo com a Seção 2.7).

Na sequência, conforme os y melhores *chunks* da etapa (vii) do *pipeline* de inferência foram verificados pelo especialista, para identificar se a recuperação encontrou documentos alternativos à resposta base (com conteúdo similar ou repetido), esses documentos foram acrescentados ao conjunto de possíveis respostas.

Considerando a necessidade de usar apenas parte do portal, foram elaboradas dez perguntas, caracterizadas na Tabela 2. Elas representam questões centrais sobre o manejo da cultura do tomate em Santa Catarina.

Essas perguntas não foram escolhidas por acaso, e ilustram diferentes aspectos da tomada de decisão de um especialista da área. De acordo com os identificadores utilizados na Tabela 2 pode-se estabelecer a ligação de cada uma delas ao assunto: A P5 aborda o manejo do tomateiro, que inicia-se antes mesmo do plantio, com a correta interpretação da análise de solo para ajustar a fertilidade, e a P1 busca informações sobre a definição de um espaçamento adequado entre as plantas para otimizar o uso da luz natural e a aeração. Após o estabelecimento da cultura, a P2 traz informações sobre o tutoramento com fitilho vertical, que é fundamental para sustentar o crescimento da planta e facilitar os tratos culturais. Já a P4 serve para trazer informações para garantir frutos maiores e mais uniformes através de um processo conhecido como raleio, que consiste na remoção do excesso de frutos. A P3 trata da irrigação por gotejamento, técnica altamente vantajosa por economizar água e reduzir a incidência de doenças ao evitar o molhamento das

Tabela 2 – Conjunto de perguntas (*queries*) utilizadas para a avaliação do pipeline de recuperação.

ID	Pergunta
P1	Como ocorre o tutoramento do tomateiro com fitilho vertical?
P2	Qual é o espaçamento mais utilizado para o tomateiro?
P3	Quais são os critérios para a interpretação de uma análise de solo para o tomateiro?
P4	Quais são os principais problemas fitossanitários causados por bactérias no tomateiro?
P5	Como deve ser realizado o monitoramento do ácaro rajado no tomateiro?
P6	Quais são os sintomas da requeima no tomateiro?
P7	Quais são as características do cultivar de tomateiro Aliança F1?
P8	Qual o objetivo do raleio de frutos no tomateiro?
P9	Quais as vantagens da irrigação por gotejamento no tomateiro?
P10	Como identificar a deficiência de cálcio (Ca) no tomateiro?

Fonte: O autor.

folhas. O sucesso da lavoura também depende de um rigoroso controle fitossanitário, que inclui o monitoramento constante de pragas, como o ácaro rajado (P8), e a atenção aos sintomas de doenças severas como a requeima (P6) e outros problemas causados por bactérias (P10). Além disso, é vital saber identificar visualmente as deficiências nutricionais, como a de cálcio (P7), que causa a podridão apical nos frutos, garantindo assim uma colheita saudável e de qualidade, sendo a escolha de um cultivar adaptado, como o Aliança F1 (P9), um fator determinante para todo o processo.

Como cada pergunta foi feita baseada em um documento, pôde-se identificar os documentos primários (as páginas com o conteúdo que pode responder perfeitamente à pergunta). Com base nos documentos que respondem cada pergunta, o especialista também forneceu uma resposta para cada pergunta (cópia literal da parte relevante do documento pertinente), criando assim o conjunto *ground truth* (dados verdadeiros).

Quanto à avaliação das respostas geradas, foram testados dois modelos de Linguagem (LLMs):

- **Meta-Llama-3-8B-Instruct**, da família de modelos Meta-Llama 3, desenvolvido pela empresa Meta AI (AI@META, 2024). Possui 8.03 bilhões de parâmetros, 32 camadas e dimensão igual a 4096.

- **gemma-7b-it**, da família de modelos Gemma, desenvolvido pela empresa Google DeepMind (TEAM et al., 2024). Possui 8.54 bilhões de parâmetros, 28 camadas e dimensão igual a 3072.

Para simplificar, ao longo do resto do trabalho referir-se-á a esses modelos, respectivamente, apenas como “modelo Llama” e “modelo Gemma”. Ambos foram escolhidos por serem *open source*, o que aumenta a reprodutibilidade deste trabalho (algo cuja relevância já se discutiu na Seção 2.8); e por serem consistentemente listados entre os melhores em sua categoria de

parâmetros. Outra razão é que em testes preliminares ambos se mostraram estáveis no ambiente do Google Colab com quantização de 4-bits, já que é preciso tanto equilibrar o uso de VRAM com o resto dos modelos usados (*embedder* e *reranker*) quanto deixar uma sobra para os picos de execução do ambiente.

Finalmente, para avaliar as respostas geradas pelo Llama e pelo Gemma, aproveita-se a existência da *ground truth* para cada uma das perguntas e portanto utiliza-se BERTScore (uma métrica que faz uso da *ground truth*). Esta abordagem foi preferida em detrimento de métricas tradicionais baseadas em sobreposição lexical (como ROUGE ou BLEU), pois o BERTScore é capaz de avaliar a similaridade semântica entre as sentenças. Além disso, métricas mais avançadas exigem o uso de mais recursos, e elas existem justamente para compensar a falta de respostas de referência.

Como a qualidade da pontuação do BERTScore depende da capacidade do PLM em avaliar corretamente no idioma alvo, utilizam-se e comparam-se dois modelos:

- **BERTimbau**: Um modelo da família BERT que foi especificamente pré-treinado para o português do Brasil (PT-BR). É um modelo especialista, cujo treinamento focado em um único idioma visa capturar com maior profundidade as nuances semânticas e estruturais da língua (SOUZA; NOGUEIRA; LOTUFO, 2020).

- **XLM-RoBERTa**: Um modelo generalista (multilíngue) de larga escala, treinado simultaneamente em 100 idiomas, incluindo o português. O modelo específico utilizado foi o xlm-roberta-large (CONNEAU et al., 2019).

Esses modelos pontuam com base na F1-score, que representa a média harmônica entre a precisão e o Cobertura calculada por esses PLMs para a similaridade entre os *tokens* (numa faixa de valores de zero a um, sendo zero similaridade mínima e um a máxima). Com o uso do BERTimbau e do XLM-RoBERTa, será possível comparar posteriormente qual é mais consistente e quais as diferenças entre as pontuações de ambos.

Como as respostas dos LLMs não são sempre as mesmas devido a sua característica estocástica, foram realizadas 30 execuções (*runs*) de cada pergunta para cada modelo, totalizando $2 \times 30 \times 10 = 600$ *runs* individuais, e depois duas avaliações (uma com BERTimbau e outra com XLM-RoBERTa) para cada *run*. Ainda, aplica-se o teste de Wilcoxon por postos assinalados (*Wilcoxon signed-rank test*) para avaliar a significância estatística das diferenças nos resultados do BERTScore (WILCOXON, 1992). A escolha deste teste se justifica por dois motivos principais:

- Natureza dos Dados: É um teste não paramétrico, o que o torna ideal para esta análise, pois não se pode presumir que as distribuições dos scores (F1, Precisão e Cobertura) sigam uma distribuição normal (Gaussiana) já que o conjunto de amostras é pequeno.

- Amostras Pareadas: O teste é projetado para amostras pareadas. Os dados obtidos se encaixam perfeitamente, pois as comparações são feitas sobre o mesmo conjunto de N

perguntas-teste (e.g., a pontuação do LLama para a Pergunta 1 é pareada com a pontuação da Gemma para a mesma Pergunta 1, para cada *run*).

O teste avalia se a mediana das diferenças entre os pares é estatisticamente diferente de zero. Para este trabalho, foi adotado um nível de significância de $\alpha = 0.05$. Um valor-p resultante inferior a $0,05$ ($p < 0,05$) é considerado evidência suficiente para rejeitar a hipótese nula, indicando que a diferença de desempenho entre os modelos é estatisticamente significativa e não provável de ter ocorrido por acaso. Como hipótese nula ($H0$), assume-se que os dois modelos não têm diferença de pontuação no BERTScore, i.e., a mediana das diferenças entre os pares de pontuações é zero.

4.2 RESULTADOS

Nessa Seção apresentam-se os resultados e suas análises preliminares: a Subseção 4.2.1 apresenta os resultados da recuperação de documentos, enquanto a Subseção 4.2.2 mostra os resultados da geração de respostas.

4.2.1 Avaliação da Recuperação de Documentos

A Tabela 3 apresenta os *chunks* trazidos pelo *pipeline* como contexto para cada pergunta, identificada pelo ID utilizado na Tabela 2. A classificação final dos documentos recuperados foi revisada pelo especialista.

Logo, o Hit Rate foi de 100%, já que para todas as perguntas o *pipeline* foi capaz de trazer ao menos um documento relevante; e a precisão foi de 100% para todas as perguntas, exceto a pergunta P4, com precisão de 60%. Observa-se que nas duas vezes em que o *pipeline* optou pelo caminho rápido, a lista de *chunks* retornados foi mais permissiva. Já o reranqueamento também foi capaz de avaliar corretamente quais os melhores *chunks*, e em 4 ocasiões (P2, P7, P8 e P10) reduziu o número de *chunks* retornados apenas aos necessários.

Outro achado importante dessa etapa do trabalho foi a maneira como o Banco de Dados vetorial organiza os dados: Pensou-se na etapa de Pré-Processamento que os “menus” trariam mais ruído do que informação, mas para responder corretamente à P4 (que requer uma lista das principais doenças bacterianas do tomate) não existe nenhum *chunk* remanescente com toda a informação. Essa informação só estava no menu hierárquico, e o *embedder* não foi capaz de inferir com boa precisão quais *chunks* se referiam a doenças bacterianas apenas a partir do título hierárquico acrescentado ao *chunk*. Na prática, os *chunks* incorretos também mencionam a palavra-chave “bactéria” ou doenças relacionadas, por isso a confusão.

Com base nesses *chunks*, são executadas as *runs* a etapa seguinte, a geração de respostas.

Tabela 3 – Chunks trazidos pelo *pipeline* como contexto logo antes da etapa (ix).

ID	Chunks Corretos	Chunks Incorretos	Caminho
P1	089_1, 89_0, 089_2, 7369_0, 2495_0		Reranqueamento
P2	85_0, 7368_0		Reranqueamento
P3	5529_0, 5529_1, 7378_0, 2509_0, 2520_1		Reranqueamento
P4	4568_2, 4569_2, 4570_2	4548_2, 5104_2	Reranqueamento
P5	4658_1, 4657_1, 2162_1, 4658_2, 4657_2		Reranqueamento
P6	4560_4, 4558_4, 4564_4, 4562_4, 67_3		Caminho Rápido
P7	979_0, 979_1		Reranqueamento
P8	02222_0, 2501_0		Reranqueamento
P9	2193_0, 7399_0, 2193_1, 2615_0, 2614_0		Caminho Rápido
P10	867_2		Reranqueamento

Fonte: O autor.

4.2.2 Análise da Geração de Respostas

Na Tabela 4 apresentam-se os resultados da média das 30 *runs* para o Llama e o Gemma, com as métricas de F1, Precisão e Cobertura calculadas pelo BERTimbau.

Já na Tabela 5, apresentam-se os resultados da média das 30 *runs* para o Llama e o Gemma, com as métricas de F1, Precisão e Cobertura calculadas pelo XLM-RoBERTa.

Observa-se que os resultados encontrados pelo BERTScore e o XLM-RoBERTa apontam uma média geral de pontuação de F1 e Precisão maior para o Gemma, e uma pontuação de Cobertura maior para o Llama. A pergunta com a pior média geral foi a P4 em ambos os avaliadores ($0,466$ e $0,809$, respectivamente), e a P5 teve a melhor média geral ($0,869$ e $0,952$, respectivamente).

Para analisar a distribuição estatística dos resultados, apresenta-se o resumo dos testes de hipótese nas Tabelas 6, 7 e 8; contendo respectivamente os resultados gerais do Teste de Wilcoxon (comparação entre o BERTScore e o XLM-RoBERTa), a comparação entre os resultados do Llama e do Gemma somente com a pontuação do XLM-RoBERTa e a comparação entre os resultados do Llama e do Gemma somente com a pontuação do BERTScore. A Hipótese Nula (H0) já foi definida anteriormente como "os dois modelos não têm diferença estatística de pontuação no BERTScore", e adotou-se um nível de significância de $\alpha = 0,05$.

Tabela 4 – Resultados Detalhados do BERTScore por Pergunta e Modelo.

	Gemma			Llama		
Pergunta	**F1**	**Precisão**	**Cobertura**	**F1**	**Precisão**	**Cobertura**
P1	0,8657	0,9425	0,8006	0,8040	0,7435	0,8819
P2	0,7686	0,8735	0,6863	0,6145	0,5472	0,7137
P3	0,6967	0,7322	0,6646	0,7684	0,7879	0,7511
P4	0,4699	0,4779	0,4623	0,5206	0,4481	0,6225
P5	0,9631	0,9408	0,9865	0,6037	0,4384	0,9694
P6	0,7877	0,9185	0,6897	0,9497	0,9604	0,9397
P7	0,7765	0,8735	0,7002	0,6439	0,6572	0,6382
P8	0,6885	0,8070	0,6004	0,8167	0,7677	0,8974
P9	0,8962	0,8989	0,8948	0,6329	0,5395	0,7676
P10	0,7640	0,9389	0,6440	0,7279	0,5880	0,9556
Média Simples	**0,7677**	**0,8404**	**0,7129**	**0,7082**	**0,6478**	**0,8137**

Fonte: O autor.

Tabela 5 – Resultados Detalhados do XLM-RoBERTa por Pergunta e Modelo.

	Gemma			Llama		
Pergunta	**F1**	**Precisão**	**Cobertura**	**F1**	**Precisão**	**Cobertura**
P1	0,9452	0,9714	0,9203	0,9129	0,8883	0,9398
P2	0,9186	0,9514	0,8881	0,8516	0,8165	0,8919
P3	0,8672	0,8829	0,8519	0,8802	0,8853	0,8753
P4	0,8107	0,8276	0,7945	0,8063	0,7917	0,8219
P5	0,9778	0,9722	0,9835	0,8520	0,7495	0,9873
P6	0,9207	0,9599	0,8847	0,9826	0,9855	0,9797
P7	0,9111	0,9441	0,8803	0,8450	0,8398	0,8513
P8	0,9110	0,9395	0,8842	0,9380	0,9144	0,9662
P9	0,9609	0,9608	0,9611	0,8594	0,8112	0,9139
P10	0,9289	0,9784	0,8841	0,9010	0,8288	0,9869
Média	**0,9152**	**0,9388**	**0,8933**	**0,8829**	**0,8511**	**0,9214**

Fonte: O autor.

Tabela 6 – Comparação Geral: Teste de Wilcoxon (BERTimbau vs. XLM-RoBERTa)

Métrica	**Valor-*p***	**Significativo** ($p < 0,05$)	**Melhor Mediana**
F1-Score	$5{,}878 \cdot 10^{-100}$	Sim	XLM-RoBERTa
Precision	$1{,}866 \cdot 10^{-99}$	Sim	XLM-RoBERTa
Cobertura	$8{,}091 \cdot 10^{-99}$	Sim	XLM-RoBERTa

Fonte: O autor.

O teste de Wilcoxon (Tabela 6) confirma que a diferença nos resultados não é aleatória. Com um valor-p de $5{,}878 \cdot 10^{-100}$ para o F1-Score, a Hipótese Nula é rejeitada com grande

Tabela 7 – Comparação Interna (XLM-RoBERTa): Teste de Wilcoxon (Llama versus Gemma)

Métrica	**Valor-*p***	**Significativo** ($p < 0,05$)	**Melhor Mediana**
F1-Score	$1,142 \cdot 10^{-14}$	Sim	Gemma
Precision	$2,370 \cdot 10^{-36}$	Sim	Gemma
Cobertura	$7,508 \cdot 10^{-17}$	Sim	LlamA

Fonte: O autor.

Tabela 8 – Comparação Interna (BERTimbau): Teste de Wilcoxon (Llama versus Gemma)

Métrica	**Valor-*p***	**Significativo** ($p < 0.05$)	**Melhor Mediana**
F1-Score	$6,754 \cdot 10^{-7}$	Sim	Gemma
Precision	$1,982 \cdot 10^{-31}$	Sim	Gemma
Cobertura	$6,414 \cdot 10^{-19}$	Sim	LlamA

Fonte: O autor.

confiança. Isso indica que o XLM-RoBERTa (multilíngue) obtém notas significativamente maiores do que o BERTScore, e pode indicar uma melhor capacidade em capturar a similaridade semântica entre as respostas geradas e o gabarito.

Em termos de qualidade de geração (F1-Score), no geral o modelo **Gemma** superou o **Llama** em ambos os cenários de avaliação, e ambas as vitórias são estatisticamente significativas:

- **Sob Avaliação do BERTimbau:** Conforme a Tabela 4, o Gemma (Média F1: 0,7677) superou o Llama (Média F1: 0,7082). A Tabela 8 também valida esta diferença como estatisticamente significativa ($p \approx 6,754 \cdot 10^{-7}$).

- **Sob Avaliação do XLM-RoBERTa:** A tendência manteve-se, como visto na Tabela 5, o Gemma (Média F1: 0,9152) superou o Llama (Média F1: 0,8829). A Tabela 7 confirma que esta diferença é significativa ($p \approx 1,142 \cdot 10^{-14}$).

Apesar da vitória geral do Gemma, os dois modelos obtiveram diferentes resultados em Precisão e Cobertura. A análise mais detalhada revela um *trade-off* claro e estatisticamente significativo no comportamento dos LLMs:

- **Gemma (Vencedor em Precisão):** O Gemma foi consistentemente melhor em **Precisão** em ambas as avaliações (Tabelas 4 e 5). Os testes de Wilcoxon (Tabelas 7 e 8) confirmam esta vitória com p-valores extremamente baixos ($p \approx 2,370 \cdot 10^{-36}$ e $p \approx 1,982 \cdot 10^{-31}$, respetivamente). Isso sugere que as respostas do Gemma eram mais corretas e continham menos informação irrelevante.

- **Llama (Vencedor em Cobertura):** Inversamente, o Llama foi consistentemente melhor em **Cobertura** em ambas as avaliações (Tabelas 7 e 8). Esta superioridade também é estatisticamente significativa ($p \approx 7,508 \cdot 10^{-17}$ e $p \approx 6,414 \cdot 10^{-19}$, respetivamente). Isso

indica que o Llama foi mais eficaz em capturar todos os pontos solicitados no gabarito, mesmo que ao custo de adicionar informação extra (reduzindo sua precisão).

Para melhor caracterizar o desempenho dos modelos, nas Tabelas 9 e 10 apresentam-se, respectivamente, alguns dos resultados obtidos pelo Llama e pelo Gemma para algumas perguntas em diferentes *runs*. Essas perguntas foram selecionadas para mostrar como a similaridade (ou diferença) entre a resposta do modelo com a *ground truth* influencia nas pontuações atribuídas pelos modelos de BERTScore.

Tanto o Gemma quanto o Llama foram capazes de trazer resultados quase perfeitos em várias perguntas, com todas as partes relevantes à resposta. Além disso, o Gemma se mostrou capaz de informar corretamente quando não existia contexto relevante para dar uma resposta completa (perceba-se que a pergunta de exemplo com baixa pontuação do Gemma é justamente a P4 discutida anteriormente). Já o Llama apresentou respostas ainda melhores e mais completas, mas esbarrou em um problema: O modelo costuma trazer respostas "muito" completas (incluindo links de referência) e extensas.

Além disso, o Llama em diversas respostas "alucina", pois fornece a resposta correta e logo após entra em *loop*, repetindo links de imagens ou afirmando erroneamente que não existe contexto relevante (sendo que logo antes ele gera uma resposta válida). Para trazer isso em números, executou-se uma busca simples pela *string* "Nenhuma informação relevante encontrada nos contextos fornecidos" (a qual o *prompt* instrui o LLM a propagar caso ele considere o contexto insuficiente) nas respostas geradas pelos dois modelos. Das 300 respostas geradas pelo Gemma, 18 explicitamente afirmaram que o contexto era insuficiente; enquanto que para as 300 respostas do Llama esse número saltou para 112. Quando o Gemma traz uma resposta com essa *string*, ela é curta e afirma claramente que não havia contexto para a resposta (todas essas eram referentes à pergunta P4), como pode-se ver na segunda pergunta da Tabela 10. Já o Llama às vezes tem esse comportamento, mas também "estraga" respostas certas acrescentando essa ou outras *strings* no final, como pode-se ver na segunda e na quarta resposta da Tabela 9: ele acrescenta o trecho "Nenhuma informação relevante encontrada [...]" logo após respostas corretas. Outrossim, as respostas com regularidade também trazem URLs de imagens e do próprio portal EpagriTec como fontes.

Justamente por essas razões o Llama tem notas menores: Por mais que ele traga respostas completas, ele acrescenta mais detalhes que o necessário, i.e., a baixa precisão é sintoma de uma dificuldade em seguir apenas as instruções do *prompt*. Em resumo, o Gemma produziu respostas com maior qualidade geral (F1-Score) ao ser mais conciso, e por consequência mais preciso; e o Llama demonstrou ser superior em garantir que toda a informação solicitada estivesse presente.

Ainda, considerando-se todos os resultados, percebe-se que o BERTScore penaliza mais as respostas com elementos incorretos e não esperados do que o XLM-RoBERTa. Isso não é necessariamente um mau comportamento, e na prática é evidência de que nenhum dos dois é perfeito: O BERTScore é mais capaz de diferenciar respostas com conteúdo diferente, enquanto

o XLM-RoBERTa é mais tolerante.

Tabela 9 – Amostra de resultados do BERTScore do Llama para XLM-RoBERTa e BERTimbau

Avaliador: BERTimbau	
Pergunta	Como deve ser realizado o monitoramento do ácaro rajado no tomateiro?
Ground Truth	O monitoramento é realizado por meio da contagem do número de ácaros de uma amostra de 20 folhas coletadas ao acaso de uma parcela (área de mesma cultivar e data de plantio). Quando forem encontrados mais de cinco ácaros por folhas em 20% das folhas amostradas, deve-se realizar o controle.
Resposta	O monitoramento é realizado por meio da contagem do número de ácaros de uma amostra de 20 folhas coletadas ao acaso de uma parcela (área de mesma cultivar e data de plantio). Quando forem encontrados mais de cinco ácaros por folhas em 20% das folhas amostradas, deve-se realizar o controle. (Fonte [...])
Scores (F1/P/R)	0,6061628461 / 0,4405722916 / 0,9711863995
Pergunta	Quais são os principais problemas fitossanitários causados por bactérias no tomateiro?
Ground Truth	Cancro Bacteriano (Clavibacter michiganensis), Mancha-bacteriana (Xanthomonas spp), Mancha-Syringae (Pseudomonas syringae pv. syringae), Murcha-bacteriana [...] Talo Oco (Pectobacterium carotovora)
Resposta	A doença mais comum causada por bactérias no tomateiro é a mancha-bacteriana (Xanthomonas spp.), [..] a doença pode ser favorecida por déficit hídrico, solo arenoso, dias curtos e adubação nitrogenada excessiva. Nenhuma informação relevante encontrada [...]. A murcha de fusarium (Fusarium oxysporum) é uma doença causada por um fungo, mas não está relacionada às bactérias. Nenhuma informação relevante encontrada [...]
Scores (F1/P/R)	0,3642 / 0,3715 / 0,3571
Avaliador: XLM-RoBERTa	
Pergunta	Quais são os sintomas da requeima no tomateiro?
Ground Truth	A requeima afeta todos os órgãos aéreos do tomateiro [...]. Nas folhas: Iniciam-se com lesão aquosa [...]. No Pecíolo/Caule: Semelhante às da folha [...]. Nos frutos: Lesões do tipo podridão dura [...]. Os sintomas severos assemelham-se a uma queimadura por geada.
Resposta	A requeima afeta todos os órgãos aéreos do tomateiro [...]. Nas folhas: Iniciam-se com lesão aquosa [...]. No Pecíolo/Caule: Semelhante às da folha [...]. Nos frutos: Lesões do tipo podridão dura [...]. Os sintomas severos assemelham-se a uma queimadura por geada.
Scores (F1/P/R)	0,9950 / 0,9991 / 0,9909
Pergunta	Quais são as características do cultivar de tomateiro Aliança F1?
Ground Truth	- Característica dos frutos: Fruto vermelho; Crescimento: Indeterminado; [...] Resistência a viroses: TSWV - Vira Cabeça [...]; Tamanho do fruto: Médio (201 a 230 g); [...] INFORMAÇÕES DA EMPRESA: [...]. RESISTÊNCIA: Fusarium oxysporum f sp. Lycopersici – (Fol 1 e 2), [...]
Resposta	O cultivar de tomateiro Aliança F1 apresenta características como fruto vermelho, crescimento indeterminado, [...] O ciclo de colheita é de 110 a 120 dias. (Nenhuma informação relevante encontrada nos contextos fornecidos.) 1 1. Nenhuma informação relevante encontrada nos contextos fornecidos. 2 [...] 1. Nenhuma informação relevante encontrada nos contextos fornecidos. 18
Scores (F1/P/R)	0.,5603 / 0,5722 / 0,5489

Fonte: O autor.

Tabela 10 – Amostra de resultados do BERTScore do Gemma para XLM-RoBERTa e BERTimbau

Avaliador: BERTimbau	
Pergunta	Como deve ser realizado o monitoramento do ácaro rajado no tomateiro?
Ground Truth	(O monitoramento é realizado por meio da contagem do número de ácaros de uma amostra de 20 folhas [...] Quando forem encontrados mais de cinco ácaros por folhas em 20% das folhas amostradas, deve-se realizar o controle.)
Resposta	O monitoramento do ácaro rajado no tomateiro é realizado por meio da contagem do número de ácaros de uma amostra de 20 folhas [...] Quando forem encontrados mais de cinco ácaros por folhas em 20% das folhas amostradas, deve-se realizar o controle.
Scores (F1/P/R)	0,9631 / 0,9408 / 0,9865
Pergunta	Quais são os principais problemas fitossanitários causados por bactérias no tomateiro?
Ground Truth	Cancro Bacteriano (Clavibacter michiganensis), Mancha-bacteriana (Xanthomonas spp), Mancha-Syringae (Pseudomonas syringae pv. syringae), Murcha-bacteriana [...] Talo Oco (Pectobacterium carotovora)
Resposta	No contexto fornecido, não há informação sobre os principais problemas fitossanitários causados por bactérias no tomateiro, logo a resposta para a pergunta é: Nenhuma informação relevante encontrada nos contextos fornecidos.
Scores (F1/P/R)	0,5168 / 0,4377 / 0,6307
Avaliador: XLM-RoBERTa	
Pergunta	Quais são os sintomas da requeima no tomateiro?
Ground Truth	A requeima afeta todos os órgãos aéreos do tomateiro [...]. Nas folhas: Iniciam-se com lesão aquosa [...]. No Pecíolo/Caule: Semelhante às da folha [...]. Nos frutos: Lesões do tipo podridão dura [...]. Os sintomas severos assemelham-se a uma queimadura por geada.
Resposta	Os sintomas da requeima no tomateiro são: lesão aquosa que cresce rapidamente, fica necrosada e com as bordas de um verde mais claro que o dos tecidos sadios. Sob condições de alta umidade, na face inferior das folhas, surge um mofo esbranquiçado. Quando o ataque é severo, pode haver coalescência das lesões, com a destruição rápida da folhagem, dando a ela um aspecto de que foi queimada por geada.
Scores (F1/P/R)	0,9260 / 0,9630 / 0,8917
Pergunta	Quais são as características do cultivar de tomateiro Aliança F1?
Ground Truth	- Característica dos frutos: Fruto vermelho; Crescimento: Indeterminado; [...] Resistência a viroses: TSWV - Vira Cabeça [...]; Tamanho do fruto: Médio (201 a 230 g); [...] INFORMAÇÕES DA EMPRESA: [...]. RESISTÊNCIA: Fusarium oxysporum f sp. Lycopersici – (Fol 1 e 2), [...]
Resposta	O cultivar de tomateiro Aliança F1 apresenta as seguintes características: * Fruto vermelho * Crescimento indeterminado * Vigor alto * Boa resistência a manchas bacterianas * Resistência a bactérias e nematoides: Mi - Meloidogyne incognita * Resistência a bactérias e nematoides: Mj - Melodogyne javanica * Resistência a fungos: [...] * Resistência a viroses: [...] * Tamanho do fruto: Médio (201 a 230 g)
Scores (F1/P/R)	0,9188 / 0,9510 / 0,8888

Fonte: O autor.

4.3 CONSIDERAÇÕES SOBRE OS RESULTADOS

A recuperação de *chunks* teve Hit Rate de 100%, com apenas dois *chunks* errados. Quanto a esse resultado, percebeu-se algo interessante: a pergunta 4 (P4) pede os principais problemas fitossanitários (doenças) causadas por bactérias. No EpagriTec, cada subitem não é identificado no corpo do texto como um problema fitossanitário causado por bactéria, essa informação fica implícita pela hierarquia do texto. A presença do título hierárquico dentro dos *chunks* foi capaz de mitigar o problema, mas não por completo. Isso demonstra que, para o portal EpagriTec, a estrutura do menu pode carregar valor semântico e sua remoção deve ser reavaliada.

5 CONSIDERAÇÕES FINAIS

O presente trabalho teve como objetivo geral desenvolver e validar uma abordagem para a recuperação e geração de respostas baseadas no conteúdo do portal EpagriTec, utilizando a técnica de Geração Aumentada por Recuperação (RAG). Para atingir esse propósito, propôs-se e implementou-se um *pipeline* completo de processamento de dados, abrangendo desde a automação da coleta de dados via *Web Scraping*, o pré-processamento, o *chunking* e a vetorização dos documentos, até a construção de um mecanismo de busca semântica com reranqueamento condicional integrado a Modelos de Linguagem em Larga Escala (LLMs), caracterizando o sistema RAG proposto. Esse fluxo permitiu transformar informações dispersas em páginas Web em uma base de conhecimento estruturada, habilitando o uso de LLMs para consulta técnica no setor agrícola.

Com base nos experimentos realizados, este trabalho responde à pergunta de pesquisa ao demonstrar que é possível extrair, formatar e estruturar dados da web para uso em um *pipeline* RAG agropecuário. Ao validar que essa transformação é possível e produz resultados consistentes, o estudo integrou os conteúdos do portal da EpagriTec a arquiteturas de RAG.

Os experimentos demonstraram a eficácia da abordagem, alcançando 100% de Hit Rate@K no conjunto de testes, indicando que o *pipeline* de recuperação e reranqueamento condicional foi capaz de encontrar ao menos um documento relevante para todas as consultas. Similarmente, a precisão de 100% para 9 das 10 perguntas indica que o *pipeline* na maior parte das vezes trouxe apenas documentos relevantes à pergunta. Na geração de respostas, observou-se um *trade-off* claro entre os LLMs avaliados: o modelo Gemma obteve maior precisão geral devido às suas respostas concisas e capacidade de seguir as instruções do *prompt*, enquanto o modelo Llama se destacou em *Recall*, capturando mais informações do gabarito, embora por vezes ao custo de "alucinações" e repetições. Percebe-se também que no geral as respostas foram compatíveis com a *ground truth* esperada.

Até o momento, o *pipeline* apresentou desempenho compatível com os recursos disponíveis, e a qualidade e eficiência dos resultados apresenta potencial de melhoria com o uso de GPUs mais modernas.

5.1 LIMITAÇÕES

A execução do *pipeline* de inferência depende de um conjunto complexo e específico de bibliotecas de IA (conforme descrito na Seção 3.4 e detalhado no código de instalação do ambiente, descrito no Apêndice A), o que pode gerar conflitos de versão e dificultar a reprodutibilidade por conta do fenômeno conhecido como "inferno de dependências" (*dependency hell*). Essas bibliotecas estão em processo de desenvolvimento e aprimoramento contínuo, e por isso foi necessário "congelar" suas versões para evitar conflitos entre as versões atualizadas e as desatualizadas.

A fonte de dados não é isenta de problemas: Assim como as bibliotecas, o site da Epa-

griTec também recebe atualizações regulares (inclusão de conteúdo, adaptação e modernização do portal) e por isso o processo de WS precisa ser refeito periodicamente. Por exemplo, no período entre a coleta dos dados e a redação final deste trabalho o portal fez uma atualização que quebra a lógica antiga do código de obtenção do título hierárquico, e uma nova coleta requeriria modificações na parte do *script* utilizada para isso.

Ainda, a avaliação das respostas dos LLMs dependeu de uma *ground truth* estática criada pelo especialista. Para um conjunto limitado de perguntas, como o utilizado nesse trabalho, isso é ótimo; mas é inviável para testar o desempenho com um número maior de perguntas, pois exigiria muita rotulação humana. Por esse motivo o conjunto de perguntas teste também foi pequeno (apenas 10).

5.2 TRABALHOS FUTUROS

Próximos passos para trabalhos futuros envolvem:

- Expandir o escopo da coleta de dados, aplicando testes para a totalidade dos conteúdos no portal EpagriTec.
- Explorar diferentes configurações dos hiperparâmetros utilizados e identificar o impacto delas na performance do *pipeline*.
- Realizar testes qualitativos com o público-alvo (técnicos da Epagri) para validar a utilidade, precisão e confiabilidade das respostas geradas pelo sistema.
- Ampliar o número de perguntas teste para no mínimo 30 perguntas, para que seja possível validar mais amplamente o desempenho do *pipeline*.

Além disso, conforme identificado na análise da Pergunta P4, a heurística de remoção de elementos padronizados eliminou os menus de navegação, e removeu acidentalmente informações contextuais valiosas, prejudicando a recuperação de listas hierárquicas. Em trabalhos futuros seria interessante avaliar os benefícios e prejuízos em não excluir os menus. Também de grande potencial é o uso de perguntas e respostas sintéticas (i.e., utilizar um LLM de grande capacidade para criar perguntas e respostas a partir de *chunks* aleatórios, coletando automaticamente informações de todos os *chunks* verdadeiros), que seria uma alternativa ao trabalho de um especialista para realizar uma bateria de testes mais extensa.

REFERÊNCIAS

AI@META. Llama 3 model card. 2024. Disponível em: <https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md>. Citado na página 46.

AKBAR, Nur Arifin. Are re-ranking in retrieval-augmented generation methods impactful for small agriculture qa datasets? a small experiment. In: **BIO Web of Conferences, 5th International Conference on Smart and Innovative Agriculture (ICoSIA 2024)**. [s.n.], 2025. v. 167, p. 01001. Disponível em: <https://doi.org/10.1051/bioconf/202516701001>. Citado 3 vezes nas páginas 27, 31 e 32.

ANNEPAKA, Yadagiri; PAKRAY, Partha. Large language models: a survey of their development, capabilities, and applications. **Knowledge and Information Systems**, v. 67, n. 3, p. 2967–3022, March 2025. ISSN 0219-3116. Disponível em: <https://doi.org/10.1007/s10115-024-02310-4>. Citado na página 12.

BASSOI, Luís Henrique et al. Agricultura de precisão e agricultura digital. **TECCOGS: Revista Digital de Tecnologias Cognitivas**, v. 20, p. 17–36, 2019. Disponível em: <http://dx.doi.org/10.23925/1984-3585.2019i20p17-36>. Citado na página 11.

BRASIL, Ministério da Agricultura e Pecuária. **Crescimento da economia brasileira é impulsionado pela alta de 15% da agropecuária em 2023**. 2024. Acesso em: 18 mar. 2025. Disponível em: <https://www.gov.br/agricultura/pt-br/assuntos/noticias/crescimento-da-economia-brasileira-e-impulsionado-pela-alta-de-15-da-agropecuaria-em-2023>. Citado na página 11.

CAMACHO-COLLADOS, Jose; PILEHVAR, Mohammad Taher. On the role of text preprocessing in neural network architectures: An evaluation study on text categorization and sentiment analysis. Association for Computational Linguistics, Brussels, Belgium, p. 40–46, nov. 2018. Disponível em: <https://aclanthology.org/W18-5406/>. Citado na página 18.

CHEN, Jianlyu et al. M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In: KU, Lun-Wei; MARTINS, Andre; SRIKUMAR, Vivek (Ed.). **Findings of the Association for Computational Linguistics: ACL 2024**. Bangkok, Thailand: Association for Computational Linguistics, 2024. p. 2318–2335. Disponível em: <https://aclanthology.org/2024.findings-acl.137/>. Citado na página 43.

CONNEAU, Alexis et al. Unsupervised cross-lingual representation learning at scale. **CoRR**, abs/1911.02116, 2019. Disponível em: <http://arxiv.org/abs/1911.02116>. Citado na página 47.

DAS, Utpal et al. Precision farming a promising technology in horticulture: A review. **International Journal of Pure & Applied Bioscience**, v. 6, n. 1, p. 1596–1606, 2018. Disponível em: <http://dx.doi.org/10.18782/2320-7051.3088>. Citado na página 11.

DEVLIN, Jacob et al. BERT: Pre-training of deep bidirectional transformers for language understanding. In: **Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)**. Minneapolis, MN, USA: Association for Computational Linguistics, 2019. p. 4171–4186. Disponível em: <https://aclanthology.org/N19-1423>. Citado 2 vezes nas páginas 19 e 20.

ELOV, Botir; KHAMROEVA, Shahlo; KHUSAINOVA, Zilola. The pipeline processing of nlp. **E3S Web of Conferences**, v. 413, 08 2023. Disponível em: <https://doi.org/10.1051/e3sconf/202341303011>. Citado na página 18.

ES, Shahul et al. RAGAs: Automated evaluation of retrieval augmented generation. In: ALETRAS, Nikolaos; CLERCQ, Orphee De (Ed.). **Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations**. St. Julians, Malta: Association for Computational Linguistics, 2024. p. 150–158. Disponível em: <https://aclanthology.org/2024.eacl-demo.16/>. Citado na página 27.

FERNÁNDEZ-PICHEL, Marcos et al. An unsupervised perplexity-based method for boilerplate removal. **Natural Language Engineering**, v. 30, n. 1, p. 132–149, 2024. Citado na página 18.

GAO, Yunfan et al. **Retrieval-Augmented Generation for Large Language Models: A Survey**. 2024. Disponível em: <https://arxiv.org/abs/2312.10997>. Citado na página 22.

GLUKHIKH, Igor N.; CHERNYSHEVA, Tatiana Y.; SHENTSOV, Yaroslav A. Decision support in a smart greenhouse using large language model with retrieval augmented generation. In: **Proc. SPIE 13217 (DTIEE 2024)**. [S.l.: s.n.], 2024. Citado 3 vezes nas páginas 28, 31 e 32.

HERLOCKER, Jonathan L. et al. Evaluating collaborative filtering recommender systems. **ACM Trans. Inf. Syst.**, Association for Computing Machinery, New York, NY, USA, v. 22, n. 1, p. 5–53, jan. 2004. ISSN 1046-8188. Disponível em: <https://doi.org/10.1145/963770.963772>. Citado na página 25.

HUMEAU, Samuel et al. **Poly-encoders: Transformer Architectures and Pre-training Strategies for Fast and Accurate Multi-sentence Scoring**. 2020. Disponível em: <https://arxiv.org/abs/1905.01969>. Citado na página 24.

IBGE, Instituto Brasileiro de Geografia e Estatística. **PIB cresce 2,9% em 2023 e fecha o ano em R$ 10,9 trilhões**. 2024. Acesso em: 18 mar. 2025. Disponível em: <https://agenciadenoticias.ibge.gov.br/agencia-sala-de-imprensa/2013-agencia-de-noticias/releases/39303-pib-cresce-2-9-em-2023-e-fecha-o-ano-em-r-10-9-trilhoes>. Citado na página 11.

JIANG, Jingchi et al. Knowledge assimilation: Implementing knowledge-guided agricultural large language model. **Know.-Based Syst.**, Elsevier Science Publishers B. V., NLD, v. 314, n. C, abr. 2025. ISSN 0950-7051. Disponível em: <https://doi.org/10.1016/j.knosys.2025.113197>. Citado 3 vezes nas páginas 27, 31 e 32.

JOHNSON, Jeff; DOUZE, Matthijs; JÉGOU, Hervé. Billion-scale similarity search with GPUs. **IEEE Transactions on Big Data**, IEEE, v. 7, n. 3, p. 535–547, 2019. Citado na página 43.

KAGERMANN, Henning; WAHLSTER, Wolfgang; HELBIG, Johannes. **Recommendations for Implementing the Strategic Initiative INDUSTRIE 4.0: Final Report of the Industrie 4.0 Working Group**. [S.l.], 2013. Disponível em: <https://en.acatech.de/publication/recommendations-for-implementing-the-strategic-initiative-industrie-4-0-final-report-of-the-industrie-4-0-working-group/>. Citado na página 11.

KHDER, Moaiad Ahmad. Web scraping or web crawling: State of art, techniques, approaches and application. **International Journal of Advanced Soft Computing and Applications**, v. 13, n. 3, p. 145–168, November 2021. ISSN 2710-1274. Accessed: 2025-03-19. Disponível em:

<https://www.researchgate.net/publication/357401723_Web_Scraping_or_Web_Crawling_State_of_Art_Techniques_Approaches_and_Application>. Citado na página 16.

LANGCHAIN. **Chunking Strategies**. 2024. <https://python.langchain.com/docs/modules/data_connection/document_transformers/text_splitters/>. Acessado em: 10 nov. 2025. Citado na página 20.

LAWSON, Richard. **Web Scraping with Python**. Birmingham: Packt Publishing Ltd, 2015. Citado na página 16.

LI, Dong et al. On sampling top-k recommendation evaluation. In: **Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining**. ACM, 2020. p. 2114–2124. Disponível em: <http://dx.doi.org/10.1145/3394486.3403262>. Citado na página 25.

LI, Jinhao et al. **Large Language Model Inference Acceleration: A Comprehensive Hardware Perspective**. 2025. Disponível em: <https://arxiv.org/abs/2410.04466>. Citado na página 16.

LIN, Chin-Yew. ROUGE: A package for automatic evaluation of summaries. In: **Text Summarization Branches Out**. Barcelona, Spain: Association for Computational Linguistics, 2004. p. 74–81. Disponível em: <https://aclanthology.org/W04-1013/>. Citado na página 26.

LIU, Nelson F. et al. **Lost in the Middle: How Language Models Use Long Contexts**. 2023. Disponível em: <https://arxiv.org/abs/2307.03172>. Citado 2 vezes nas páginas 19 e 20.

MA, Hanjie et al. Sdd-lawllm: Advancing intelligent legal systems through synthetic data-driven fine-tuning of large language models. **Electronics**, v. 14, n. 4, 2025. ISSN 2079-9292. Disponível em: <https://www.mdpi.com/2079-9292/14/4/742>. Citado 3 vezes nas páginas 29, 31 e 32.

MANNING, Christopher D.; RAGHAVAN, Prabhakar; SCHüTZE, Hinrich. **Introduction to Information Retrieval**. Cambridge, UK: Cambridge University Press, 2009. Online edition draft April 1, 2009. Disponível em: <https://nlp.stanford.edu/IR-book/pdf/irbookonlinereading.pdf>. Citado na página 25.

MINAEE, Shervin; MIKOLOV, Tomas et al. Large language models: A survey. **Artificial Intelligence Review**, 2024. Preprint (ainda não revisado por pares). Acessado em: 9 de junho de 2025. Disponível em: <https://doi.org/10.48550/arXiv.2402.06196>. Citado na página 15.

MITCHELL, Ryan. **Web Scraping with Python: Collecting Data from the Modern Web**. 2nd. ed. Sebastopol, CA: O'Reilly Media, 2018. ISBN 978-1491985571. Citado 3 vezes nas páginas 16, 17 e 18.

MORALES, Rafael Gustavo Ferreira; HICKEL, Eduardo. Eficiência da plataforma epagritec como fonte de informações técnicas na área agrícola. In: **Práticas e Pesquisas Aplicadas em Ciências Exatas**. e-Publicar, 2023. p. 124. Apresentado no Congresso Brasileiro de Agronomia (CBA 2021). Disponível no ResearchGate. Disponível em: <https://www.researchgate.net/publication/374750197_EFICIENCIA_DA_PLATAFORMA_EPAGRITEC_COMO_FONTE_DE_INFORMACOES_TECNICAS_NA_AREA_AGRICOLA>. Citado na página 34.

MURTAZA, Syed Shariyar et al. Implementing retrieval augmented generation technique on unstructured and structured data sources in a call center of a large financial institution. In: **Proceedings of the North American Chapter of the Association for Computational Linguistics (NAACL) – Industry Track**. [S.l.: s.n.], 2025. p. 598–606. Citado 2 vezes nas páginas 19 e 23.

MUSTAPHA, Shehu et al. A demystified overview of data scraping. **International Journal of Data Science and Advanced Analytics**, v. 6, n. 1, p. 290–296, Jun. 2024. Disponível em: <https://ijdsaa.com/index.php/welcome/article/view/205>. Citado na página 17.

OLIVEIRA, Rosana Cavalcante de; SILVA, Rogério Diogne de Souza e. Artificial intelligence in agriculture: Benefits, challenges, and trends. **Applied Sciences**, v. 13, n. 13, 2023. ISSN 2076-3417. Disponível em: <https://www.mdpi.com/2076-3417/13/13/7405>. Citado na página 11.

PAN, James Jie; WANG, Jianguo; LI, Guoliang. Vector database management techniques and systems. In: **Companion of the 2024 International Conference on Management of Data**. New York, NY, USA: Association for Computing Machinery, 2024. (SIGMOD '24), p. 597–604. ISBN 9798400704222. Disponível em: <https://doi.org/10.1145/3626246.3654691>. Citado 2 vezes nas páginas 21 e 22.

PANOUTSOPOULOS, Hercules et al. Investigating the effect of different fine-tuning configuration scenarios on agricultural term extraction using bert. **Computers and Electronics in Agriculture**, v. 225, p. 109268, 2024. ISSN 0168-1699. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0168169924006598>. Citado 3 vezes nas páginas 27, 31 e 32.

PAPINENI, Kishore et al. Bleu: a method for automatic evaluation of machine translation. In: ISABELLE, Pierre; CHARNIAK, Eugene; LIN, Dekang (Ed.). **Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics**. Philadelphia, Pennsylvania, USA: Association for Computational Linguistics, 2002. p. 311–318. Disponível em: <https://aclanthology.org/P02-1040/>. Citado na página 26.

PERSSON, Emil. **Evaluating tools and techniques for web scraping**. Dissertação (Degree Project in Computer Science and Engineering, Second Cycle) — KTH Royal Institute of Technology, Stockholm, Sweden, 2019. Acessado em: 12 de junho de 2025. Disponível em: <https://www.diva-portal.org/smash/record.jsf?pid=diva2%3A1345238>. Citado na página 16.

RAMYA, C; SHREEDHARA, KS; KAVITHA, G. Preprocessing: A prerequisite for discovering patterns in wum process. **Computing Research Repository - CORR**, 04 2011. Citado na página 17.

REIMERS, Nils; GUREVYCH, Iryna. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In: INUI, Kentaro et al. (Ed.). **Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)**. Hong Kong, China: Association for Computational Linguistics, 2019. p. 3982–3992. Disponível em: <https://aclanthology.org/D19-1410/>. Citado 2 vezes nas páginas 20 e 21.

REZAYI, Saed et al. Agribert: Knowledge-infused agricultural language models for matching food and nutrition. In: RAEDT, Lud De (Ed.). **Proceedings of the Thirty-First International**

Joint Conference on Artificial Intelligence, IJCAI-22. International Joint Conferences on Artificial Intelligence Organization, 2022. p. 5150–5156. AI for Good. Disponível em: <https://doi.org/10.24963/ijcai.2022/715>. Citado 3 vezes nas páginas 27, 31 e 32.

SAMUEL, Dinesh Jackson et al. **AgroLLM: Connecting Farmers and Agricultural Practices through Large Language Models for Enhanced Knowledge Transfer and Practical Application**. 2025. Disponível em: <https://arxiv.org/abs/2503.04788>. Citado na página 29.

SELLAM, Thibault; DAS, Dipanjan; PARIKH, Ankur. BLEURT: Learning robust metrics for text generation. In: JURAFSKY, Dan et al. (Ed.). **Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics**. Online: Association for Computational Linguistics, 2020. p. 7881–7892. Disponível em: <https://aclanthology.org/2020.acl-main.704/>. Citado na página 26.

SOHN, Jiwoong et al. Rationale-guided retrieval augmented generation for medical question answering. In: **Proc. NAACL 2025**. [S.l.: s.n.], 2025. p. 12739–12753. Citado 3 vezes nas páginas 29, 31 e 32.

SOUZA, Fábio; NOGUEIRA, Rodrigo; LOTUFO, Roberto. BERTimbau: pretrained BERT models for Brazilian Portuguese. In: **9th Brazilian Conference on Intelligent Systems, BRACIS, Rio Grande do Sul, Brazil, October 20-23 (to appear)**. [S.l.: s.n.], 2020. Citado na página 47.

TARANNUM, Tasnuva. **Cleaning of Web Scraped Data with Python**. Tese (Doutorado) — Brac University, apr 2019. Dissertação (B.Sc. CSE). Disponível em: <http://hdl.handle.net/10361/12354>. Citado na página 17.

TEAM, Gemma et al. **Gemma: Open Models Based on Gemini Research and Technology**. 2024. Disponível em: <https://arxiv.org/abs/2403.08295>. Citado na página 46.

THAKUR, Nandan et al. **BEIR: A Heterogenous Benchmark for Zero-shot Evaluation of Information Retrieval Models**. 2021. Disponível em: <https://arxiv.org/abs/2104.08663>. Citado na página 24.

UPADHYE, Akshata. A comprehensive survey of text data cleaning techniques: Challenges, methods, and best practices. **Journal of Scientific and Engineering Research**, v. 7, n. 8, p. 205–210, 2020. Citado na página 18.

VAGHEFI, Saeid Ashraf et al. Chatclimate: Grounding conversational ai in climate science. **Communications Earth & Environment**, v. 4, p. 480, 2023. Citado 3 vezes nas páginas 29, 31 e 32.

VASWANI, Ashish et al. Attention is all you need. In: **Advances in Neural Information Processing Systems (NeurIPS)**. [s.n.], 2017. v. 30. Disponível em: <https://arxiv.org/abs/1706.03762>. Citado na página 15.

Weaviate. **Chunking Strategies to Improve Your RAG Performance**. 2025. <https://weaviate.io/blog/chunking-strategies-for-rag>. Acesso em: 12 nov. 2025. Citado na página 19.

WEERASINGHE, Manushi. Enhancing web scraping with artificial intelligence: A review. In: KOTELAWALA DEFENCE UNIVERSITY. **Proceedings of the 4th Research Symposium of the Faculty of Computing**. 2024. Acessado em: 8 de junho de 2025. Disponível em:

<https://www.researchgate.net/publication/379024314_Enhancing_Web_Scraping_with_Artificial_Intelligence_A_Review>. Citado na página 17.

WILCOXON, Frank. Individual comparisons by ranking methods. In: ____. **Breakthroughs in Statistics: Methodology and Distribution**. New York, NY: Springer New York, 1992. p. 196–202. ISBN 978-1-4612-4380-9. Disponível em: <https://doi.org/10.1007/978-1-4612-4380-9_16>. Citado na página 47.

XIONG, Jie et al. Enhancing plant protection knowledge with large language models: A fine-tuned qa system using lora. **Applied Sciences**, v. 15, n. 7, p. 3850, 2025. Citado 3 vezes nas páginas 28, 31 e 32.

YANG, Shuting et al. Shizishangpt: An agricultural large language model integrating tools and resources. **arXiv preprint arXiv:2409.13537**, 2024. Citado na página 29.

ZHANG, Tianyi et al. **BERTScore: Evaluating Text Generation with BERT**. 2020. Disponível em: <https://arxiv.org/abs/1904.09675>. Citado na página 26.

ZHANG, Zhihao et al. Beefbot: Harnessing advanced LLM and RAG techniques for providing scientific and technology solutions to beef producers. In: **Proceedings of the 31st International Conference on Computational Linguistics: System Demonstrations (COLING Demos 2025)**. Abu Dhabi, UAE: Association for Computational Linguistics, 2025. p. 54–62. Disponível em: <https://aclanthology.org/2025.coling-demos.7/>. Citado 3 vezes nas páginas 28, 31 e 32.

ZHAO, Wayne Xin et al. A survey of large language models. 2025. Disponível em: <https://arxiv.org/abs/2303.18223>. Citado na página 15.

APÊNDICE A – *SCRIPT* DE INSTALAÇÃO DAS BIBLIOTECAS

Este apêndice contém a lista exata de dependências utilizadas na configuração do Google Colab para que todas as bibliotecas funcionem juntas.

```
1 !pip install --upgrade --quiet \
2 "langchain==1.0.0a10" \
3 "langchain-community==0.3.31" \
4 "langchain-core==0.4.0.dev0" \
5 "langchain-google-genai==2.1.12" \
6 "langchain-openai==0.3.35" \
7 "langchain-text-splitters==0.3.11" \
8 "langchain-huggingface==0.3.1" \
9 "langchain-chroma==0.2.6" \
10 "transformers==4.57.1" \
11 "pydantic==2.12.2" \
12 "pydantic-settings==2.11.0" \
13 "pydantic_core==2.41.4" \
14 "accelerate==1.10.1" \
15 "bitsandbytes==0.48.1" \
16 "peft==0.17.1" \
17 "sentence-transformers==5.1.1" \
18 "faiss-cpu==1.12.0" \
19 "chromadb==1.2.0" \
20 "datasets==4.2.0" \
21 "tokenizers==0.22.1" \
22 "huggingface-hub==0.35.3" \
23 "torch==2.8.0+cu126" \
24 "torchaudio==2.8.0+cu126" \
25 "torchvision==0.23.0+cu126" \
26 "pandas==2.2.2" \
27 "numpy==1.26.4" \
28 "scikit-learn==1.6.1" \
29 "scipy==1.16.2" \
30 "matplotlib==3.10.0" \
31 "seaborn==0.13.2" \
32 "plotly==5.24.1" \
33 "tqdm==4.67.1" \
34 "requests==2.32.5" \
35 "bert-score==0.3.13"
```